

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ

**«Национальный исследовательский университет ИТМО»
(Университет ИТМО)**

Факультет Программной инженерии компьютерной техники

**Образовательная программа: Системное и прикладное программное
обеспечение 2025**

Направление подготовки (специальность): 09.04.04 – Программная инженерия

О Т Ч Е Т

о Научно–исследовательской работе

Тема задания: Обзор предметной области тематики выпускной квалификационной работы

Обучающийся: Терешкин Дмитрий Александрович, Р4114

Научный руководитель: Штенников Дмитрий Геннадьевич, старший преподаватель

Ответственный за практику: Маркина Татьяна Анатольевна, старший преподаватель

Дата ____

Санкт–Петербург
2026

ВВЕДЕНИЕ

Тема НИР: методы мультимодального анализа визуальных и акустических данных для автоматизации обнаружения БПЛА.

Актуальность. Распространение беспилотных летательных аппаратов (БПЛА) привело к росту угроз безопасности критически важных объектов инфраструктуры, массовых мероприятий и воздушного движения. Существующие унимодальные решения нестабильны в сложных условиях эксплуатации: визуальные детекторы теряют качество при малом угловом размере объекта, сложном фоне, деградации изображения (дальность, погода, освещённость) и склонны к ложным срабатываниям на похожих объектах (птицы, мелкие летающие предметы); акустические методы менее чувствительны к освещённости, но резко зависят от уровня фонового шума. Универсальной «одной» модальности для надёжного обнаружения в реальных условиях нет; совмещение разнородных сенсоров – одна из ключевых тенденций повышения устойчивости систем *anti-UAV*. Это обосновывает разработку мультимодального решения, объединяющего визуальный и акустический каналы. Обоснование опирается на аналитический обзор предметной области, выполненный в рамках НИР 1 семестра (см. раздел 1).

Цель НИР: спроектировать систему мультимодального (визуальное и акустическое) обнаружения БПЛА, обосновать модели, методы, технологии и оборудование, разработать графические и математические модели создаваемого продукта, спланировать испытания и провести пилотные исследования; подготовить отчёт о результатах работы и черновик статьи в журнал РИНЦ.

Задачи НИР (по этапам рабочего плана-графика) представлены в таблице 1.

Таблица 1 – Задачи НИР по этапам рабочего плана-графика

№ этапа	Название этапа	Длит., дн.	Задание этапа
1	Инструктаж обучающегося	1	Инструктаж по требованиям охраны труда, техники безопасности, пожарной безопасности, правилам внутреннего трудового распорядка
2	Модели и методы для решения задач исследования	11	Определение моделей и методов для решения, поставленных задач исследования
3	Выбор и обоснование технологий и оборудования	22	Выбор и обоснование технологий и оборудования для реализации проекта
4	Описание проекта создаваемого продукта	22	Описание проекта создаваемого продукта для проведения исследования графическими и математическими моделями

№ этапа	Название этапа	Длит., дн.	Задание этапа
5	План испытаний продукта, перечень ожидаемых результатов	17	План тестирования продукта, перечень ожидаемых результатов его применения на практике, проведение пробных замеров
6	Проведение пилотных исследований	46	Проведение пилотных исследований, корректировка плана исследований, обработка и интерпретация результатов
7	Отчёт по НИР, статья РИНЦ	20	Подготовка отчёта о результатах НИР, получение отзыва руководителя (ГОСТ 7.32-2017); написание черновика статьи в журнал РИНЦ

1 СОСТОЯНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ (КРАТКИЙ АНАЛИТИЧЕСКИЙ ОБЗОР)

1.1 Методы обнаружения БПЛА – таксономия

Методы систематизируются по двум осям:

- сенсорная модальность (видео / аудио / мультимодальность);
- уровень интеграции данных (уровень признаков / уровень решений / гибрид).

Визуальные методы (компьютерное зрение). Систематизация подходов и обзоры состояния области детекции и классификации БПЛА приведены в [19, 20, 23]. Одностадийные детекторы (приоритет реального времени) – семейство *YOLO* и *EfficientDet* [22] (*BiFPN* и *compound scaling*). Семейство *YOLO* исторически развивается от первой версии [15], где детекция формулировалась как единая регрессионная задача, и её многомасштабного расширения *YOLOv3* [16], к последующим поколениям с более лёгкими архитектурами и улучшениями для малых объектов (адаптации линейки *YOLOv5* под малые цели рассматриваются в *anti-UAV*-работах [3, 6, 8, 13]), вплоть до *YOLOv8* – современного поколения с улучшениями архитектуры, функции потерь и процедуры обучения. **Применительно к настоящей работе** в качестве визуального детектора рассматривается именно *YOLOv8* (см. раздел 2.2.1); более ранние версии *YOLO* здесь упоминаются только в обзорном контексте – для понимания эволюции семейства. Двухстадийные детекторы (*Faster R-CNN* с *Region Proposal Network* [18]) дают более высокую точность ценой скорости и используются как референс «верхней границы» качества. Сквозная проблема – детекция малоразмерных объектов: БПЛА занимает считанные десятки пикселей на дальних дистанциях [19, 23]; решается модификациями *neck/backbone*, *attention*-модулями, повышением разрешения входа, настройкой якорей, аугментацией [3, 6, 8, 13].

Акустические методы. Эволюция: от *MFCC*/спектральных признаков с классическими классификаторами (*SVM*, *Random Forest*) – интерпретируемый и вычислительно дешёвый *baseline* [7] – к спектрограммам с *CNN* (в том числе *lightweight CNN*, ориентированными на встраиваемые системы и реальное время [2], а также к глубокому распознаванию в сложных средах [14]) и адаптивным механизмам внимания (*attention/cross-attention*) для подавления нерелевантных временных сегментов [17]. Ключевые проблемы – робастность к шуму городской среды [14] и ограниченность публичных акустических датасетов [12].

Мультимодальное слияние (*fusion*). Стандартная таксономия [10]: раннее слияние (на уровне признаков) – высокий потенциал точности за счёт кросс-модальных зависимостей, но жёсткие требования к синхронизации и вычислительным ресурсам [10,

11]; позднее слияние (на уровне решений) – проще по реализации, устойчиво к асинхронности модальностей, гарантирует выполнение требования реального времени, позволяет явную компенсацию ошибок – подавление ложных срабатываний при отсутствии подтверждения второй модальностью [4, 9, 10]; гибридное – комбинация уровней, практичный компромисс [1, 5, 24]. Сквозные темы – временное и семантическое выравнивание модальностей (*alignment*) [11] и устойчивость к отказу/отсутствию одной из модальностей (*missing modality*) [25].

1.2 Критический анализ и «белые пятна»

Сопоставление подходов из 1.1 показывает, что в *anti-UAV*-литературе закрыты в основном «качественные» аспекты (точность отдельных детекторов на «чистых» данных), тогда как ряд важных для практики аспектов остаётся систематически недостаточно проработанным. Ниже перечислены те пробелы, которые **прямо влияют на цель настоящей работы** и в дальнейшем определяют состав плана испытаний (раздел 5) и интерпретацию пилотных результатов (раздел 6):

- **Временное выравнивание аудио и видео** – методологически известная задача (видео обрабатывается покадрово, аудио – оконно) [11], но прикладные *anti-UAV*-работы часто не описывают явный механизм синхронизации и не измеряют межмодальную задержку.
- **Полевые испытания и реалистичные шумовые условия** – датасеты преимущественно лабораторные, переносимость результатов не гарантирована; ощущается дефицит реальных акустических данных [12, 14].
- **Метрики вычислительных ресурсов и энергопотребления** – качество (точность) сообщают часто, ресурсы (число параметров, *FLOPs*, память, энергопотребление) для развёртывания на ограниченном оборудовании – редко [4, 5, 10].
- **Устойчивость к отсутствующей модальности** – обсуждается в обзорных работах [25], но в прикладных *anti-UAV*-работах постановка задачи пока редка.
- **Защищённость от намеренных помех / маскировки (*adversarial / spoofing*)** – упоминается как практический вызов [21], конкретные методы противодействия представлены слабо.

Итог критического анализа. Перечисленные пробелы определяют требования к будущей системе: явная модель времени и измерение Δt (пробел 1), воспроизводимое исследование устойчивости в условиях, приближённых к реальным (пробелы 2 и 4 – решаются «каналом деградации», раздел 4.2.3), обязательный отчёт о вычислительных ресурсах наряду с точностью (пробел 3), модульность «выключения» модальности как

штатный режим работы (пробел 4). Пробел 5 (защищённость от помех) выходит за рамки настоящей НИР и обозначен как направление для магистерской диссертации.

1.3 Рекомендованный подход (по итогам обзора)

На основе сопоставления подходов (1.1) и выявленных пробелов (1.2) сформирована стартовая архитектура – компромисс между точностью, скоростью и реализуемостью на доступном оборудовании. Стартовая архитектура: визуальная ветка – оптимизированный детектор семейства *YOLO* для малых целей [3, 6, 8, 13]; акустическая ветка – *lightweight CNN* по спектрограммам/*MFCC* [2]; стратегия слияния – начать с позднего слияния на уровне уверенностей (гарантирует $\geq 20\text{ FPS}$, прозрачную компенсацию ложных срабатываний) [4, 9, 10], затем перейти к гибриднему при наличии данных и вычислительного бюджета [10, 11]; механизм адаптивного доверия к модальностям (взвешивание по *SNR* аудио / качеству изображения) [17]; явная модель времени для синхронизации модальностей [11].

Целевые показатели, которыми измеряется успех решения и которые далее (раздел 5) операционализируются в плане испытаний: прирост точности обнаружения на 10–15 % относительно лучшего унимодального решения при производительности не ниже 20 *FPS*, снижение *false positive/false negative* за счёт компенсации ошибок между каналами, контролируемая (предсказуемая) межмодальная задержка Δt , плавная деградация качества при отказе одной из модальностей. Конкретизация архитектурных решений и используемых моделей – в разделе 2.

2 МОДЕЛИ И МЕТОДЫ ИССЛЕДОВАНИЯ

2.1 Постановка задачи

Рассматривается задача **бинарного обнаружения БПЛА** в потоке разнородных данных.

Вход системы: два синхронизируемых потока – видеопоток $V = \{(x_n, t_n)\}$ (кадры x_n с временными отметками t_n , частота f_v) и аудиопоток $A = \{(a_m, \tau_m)\}$ (аудио-окна a_m длины T_a с шагом h_a и временными отметками τ_m). Каждая модальность может быть активна или отсутствовать (отказ/недоступность сенсора).

Выход системы: поток решений $D = \{d_k\}$, где $d_k = \langle t_k, y_k, c_k, b_k?, id_k? \rangle$: - t_k – временная отметка события; - $y_k \in \{\text{«БПЛА»}, \text{«не-БПЛА»}\}$ – бинарное решение; - $c_k \in [0,1]$ – итоговая оценка уверенности; - b_k – ограничивающая рамка объекта (только при наличии визуального канала, опционально); - id_k – идентификатор трека объекта (только при наличии визуального канала и трекера, опционально).

Объекты-помехи (источники ложных срабатываний), которые система должна отличать от БПЛА: птицы, самолёты и другие летающие объекты в видеоканале; фоновый шум (транспорт, ветер, индустриальные источники, голоса) в аудиоканале.

Режим работы – потоковый (*event-driven*): обработка выполняется онлайн, по мере поступления кадров и аудио-окон; решения формируются с задержкой не более заданной (требование реального времени).

Критерии решения задачи (формальные определения и целевые значения – раздел 6): качество обнаружения (*precision, recall, F1*; для визуального канала – также *mAP@IoU*), производительность (*FPS, end-to-end latency*), вычислительные ресурсы (число параметров, *FLOPs*, потребление памяти), межмодальная задержка Δt , устойчивость к деградации каналов и к отсутствию одной из модальностей. Ключевой критерий – прирост *F1* от мультимодального слияния относительно лучшего унимодального решения при сохранении производительности $\geq 20 FPS$.

2.2 Выбранные модели и методы

В рамках исследования по итогам аналитического обзора (раздел 1) и постановки задачи (2.1) был сформирован конкретный набор моделей и методов, реализующих рекомендованный подход (1.3). Ниже описано, что именно выбрано в каждой ветке, откуда взято и почему пригодно для нашей цели – обеспечить прирост точности от мультимодального слияния при сохранении производительности реального времени.

2.2.1 Визуальная ветка

В качестве визуального детектора в настоящей работе выбран детектор объектов *YOLOv8* [26] (реализация *Ultralytics*) – современное поколение семейства *YOLO* [15, 16], развивающее линию адаптации одностадийных детекторов под малоразмерные цели [3, 6, 8, 13]. Этот выбор является развитием рабочего пайплайна бакалаврской ВКР [36], где *YOLOv8* на комбинированном датасете «дрон/птица/самолёт» показал $precision = 0.85$, $recall = 0.79$, $mAP50 = 0.83$, $mAP50-95 = 0.54$ при работе в реальном времени. Выбор *YOLOv8* обоснован: (а) наличием отлаженного пайплайна и опыта работы; (б) высокой точностью на малоразмерных объектах при производительности реального времени; (в) удобной интеграцией с алгоритмами трекинга.

Для устойчивости детекции во времени интегрируется алгоритм трекинга *ByteTrack* [27] – он использует все детекции, включая низкоуверенные (двухэтапная ассоциация: сначала высокоуверенные детекции с треками по IoU , затем оставшиеся треки с низкоуверенными детекциями), что повышает устойчивость в сложных условиях; в ВКР обеспечил 31.3 *FPS* (реальное время) при средней длительности трека 517.6 кадра [36]. Трекер используется для сглаживания детекций, сохранения идентификаторов объектов между кадрами и (как направление развития) для обратной связи «трекер → детектор» – использования информации о траектории для корректировки параметров детектирования.

В обзоре альтернатив – для понимания «верхней границы» качества и масштабируемых аналогов – рассматривались также *Faster R-CNN* [18] (референс точности ценой скорости) и *EfficientDet* [22] (масштабируемая альтернатива одностадийных детекторов); они не выбраны как основное решение по тем же причинам, по которым выбран *YOLOv8* (приоритет реального времени и наличие задела).

Выбор *YOLOv8* выполнен не только из-за наличия готового пайплайна, но и по набору инженерных критериев: (1) выполнение требования реального времени не ниже 20 *FPS* на целевой *GPU GTX 1660 Ti*; (2) качество на малых объектах за счёт многоуровневой обработки признаков; (3) воспроизводимый стек *Ultralytics* с готовыми средствами обучения, валидации и экспорта; (4) простая интеграция с *ByteTrack* и существующим *Kafka*-пайплайном; (5) минимальная стоимость сопровождения по сравнению с двухстадийными детекторами.

Критерий	<i>YOLOv8n</i> / <i>YOLOv8s</i>	Альтернативы	Вывод для проекта
Скорость инференса	Одностадийная архитектура; в пилоте 9–13 мс/кадр на <i>GTX 1660 Ti</i> .	<i>Faster R-CNN</i> обычно точнее, но дороже по <i>latency</i> ; <i>EfficientDet</i> требует отдельной настройки масштаба.	<i>YOLOv8</i> лучше соответствует ограничению $FPS \geq 20$.

Малые объекты	Многоуровневые признаки и удобная настройка размера входа <i>imgsz</i> ; хорошо стыкуется с аугментациями.	<i>YOLOv5</i> близок, но в работе уже выбран более новый пайплайн; <i>Faster R-CNN</i> можно оставить как референс качества.	Для дальних БПЛА приоритетна скорость при приемлемом <i>mAP</i> .
Интеграция	Поддерживает стандартный <i>bbox-output, conf, class_id</i> ; напрямую подключается к <i>ByteTrack</i> .	Для <i>EfficientDet/Faster R-CNN</i> потребовались бы дополнительные адаптеры форматов и профилирование.	Снижается риск ошибок интеграции в потоковом стенде.
Воспроизводимость	Есть документация, <i>CLI/API</i> обучения и выгрузка <i>metrics/results.csv</i> .	Альтернативы требуют больше самостоятельной обвязки.	Проще показать метрики обучения и повторить эксперимент.

Итого: *YOLOv8* используется как основной визуальный детектор, а *Faster R-CNN* и *EfficientDet* оставлены в работе как ориентиры для сравнения качества/скорости, но не как базовая реализация из-за риска нарушения требования реального времени.

2.2.2 Акустическая ветка

В качестве акустического детектора выбрана облегчённая свёрточная нейросеть (*lightweight CNN*) по спектрограммам/*MFCC* [2] – современный *baseline* для бинарной задачи «дрон / не-дрон» по аудио. На входе сети – спектрограммы или мел-частотные кепстральные коэффициенты (*MFCC*); конкретный тип признаков (*mfcc / mel*-спектрограмма) задаётся параметром (раздел 3.2 – *feature* в конфигурации сервиса). Выбор обоснован ориентацией на низкую латентность и работу на ограниченном оборудовании (модель «целевого» *edge*-устройства) [2], а также эмпирическими данными из литературы об эффективности *MFCC* при низком отношении сигнал/шум [14]. Как направление развития рассматривается архитектура с механизмами внимания (*cross-attention*, адаптивное взвешивание признаков) для подавления нерелевантных временных сегментов и повышения робастности к шуму [17].

Замечание о реализации. На этапе пилотных исследований дополнительно был апробирован альтернативный *backend* – *Audio Spectrogram Transformer (AST)* с предобучением на крупном корпусе *AudioSet* и дообучением на акустических наборах БПЛА (раздел 6); его подключение в архитектуре предусмотрено как сменный *backend* акустического детектора и не изменяет верхнеуровневую структуру пайплайна.

2.2.3 Стратегия слияния

Применительно к настоящей работе в качестве стартовой стратегии выбрано позднее слияние (*late fusion*) на уровне уверенностей [4, 9, 10]: каждая ветка формирует собственное

решение и оценку уверенности, оценки агрегируются (взвешенная сумма с правилом компенсации ложных срабатываний – формальное определение в разделе 4.2). Выбор обоснован: гарантией выполнения требования $\geq 20 \text{ FPS}$ (модальности обрабатываются независимо, могут параллелиться), устойчивостью к асинхронности и «плавающей» задержке аудио-окон, прозрачностью (понятная компенсация *false positive / false negative*), простотой реализации [4, 9, 10].

На вторую итерацию предусмотрен переход к гибридному слиянию при наличии данных и вычислительного бюджета [1, 5, 10, 11, 24]. Чтобы реализовать рекомендацию обзора об «адаптивном доверии к модальностям» (раздел 1.3), вводится механизм адаптивного взвешивания – динамические веса w_v, w_a в зависимости от качества канала (*SNR* аудио, резкость/освещённость кадра): когда канал «надёжнее», его вклад в итоговое решение больше (идея адаптивного взвешивания признаков переносится из [17]). Чтобы решить проблему «временного выравнивания» (пробел 1 из 1.2), вводится явная модель времени [11]: видео покадровое, аудио оконное; сопоставление событий по перекрытию временных интервалов в окне $[t - \varepsilon, t + \varepsilon]$.

2.2.4 Режимы работы (модульность)

Чтобы обеспечить устойчивость к отсутствующей модальности (пробел 4 из 1.2) и поддержать применимость системы при разном составе доступного оборудования, в системе реализуются четыре режима, переключаемых конфигурацией: *video-only* (только визуальная ветка), *audio-only* (только акустическая), *late fusion* (обе ветки, позднее слияние [10]), *hybrid fusion* (обе ветки, гибридное слияние [10, 11]). Эти же режимы служат основой для *ablation*-экспериментов (раздел 5) – прямого измерения вклада слияния относительно лучшего унимодального решения.

2.3 Метрики и протоколы оценки

Для оценки решения используется следующий набор метрик и протоколов:

- Качество обнаружения – *precision, recall, F1*; для визуальной ветки дополнительно *mAP@IoU* (стандартная метрика детекции объектов, в т. ч. для семейства *YOLO* [15, 16] и *Faster R-CNN* [18]).
- Качество трекинга – *IDF1*, число переключений идентификаторов (*ID-switch*) – стандартные метрики многообъектного трекинга, применявшиеся в том числе при оценке *ByteTrack* [27].
- Производительность – *FPS* (число обработанных кадров в секунду), *end-to-end latency* (время от поступления кадра/аудио-окна до выдачи решения, включая прохождение через брокера сообщений [28]).

- Вычислительные ресурсы – число параметров модели, *FLOPs*, потребление памяти (отдельный замер ресурсов восполняет один из пробелов литературы – раздел 1.2 [4, 5, 10]).
- Межмодальная задержка – Δt (разность времён сопоставленных детекций модальностей [11]). (4.7)
- Ключевой эксперимент – *ablation*-сравнение четырёх режимов (*video-only* / *audio-only* / *late* / *hybrid*) и оценка устойчивости к контролируемым искажениям потока (раздел 5).

3 ТЕХНОЛОГИИ И ОБОРУДОВАНИЕ ДЛЯ РЕАЛИЗАЦИИ ПРОЕКТА

3.1 Архитектура реализации – потоковый пайплайн на брокере сообщений

Система реализуется как потоковый (*event-driven*) пайплайн на брокере сообщений *Apache Kafka* [28]. Источник данных – сменный: на текущем этапе используется имитатор источников данных (воспроизведение записей из публичных датасетов [12, 30, 31, 32]; в перспективе – синтетически сгенерированные сцены), в дальнейшем – реальные сенсоры (камера, микрофон).

Топология (продюсеры → топики → консьюмеры):

- Источник данных (*Data Source Adapter*) публикует кадры и аудио-окна с временными отметками в топики *video.raw* и *audio.raw*. Опционально между источником и топиками включается «канал деградации» – модуль, вносящий параметризуемые искажения потока (сдвиг аудио относительно видео Δt , дроп кадров, добавление шума в аудио, временное отключение канала) для проведения стресс-экспериментов.
- Детекторы-консьюмеры: визуальный (*YOLOv8* [26] и *ByteTrack* [27]) потребляет *video.raw*, акустический (*lightweight CNN* [2]) потребляет *audio.raw*; оба публикуют результаты (метка, уверенность, латентность; для визуального – также рамка и идентификатор трека) в топик *inference*. Каждый детектор запускается опционально – отсюда модульность режимов.
- Модуль слияния (*Fusion engine*) потребляет *inference*, выполняет временное выравнивание (накапливает окно последних детекций по временным отметкам [11]), позднее [4, 9, 10] или гибридное [10, 11] слияние с адаптивным взвешиванием [17], публикует итоговые решения в топик *decisions*.
- *Sink / Dashboard* потребляет *decisions*, ведёт лог, автоматически собирает метрики, визуализирует результаты.

Обоснование выбора потоковой архитектуры на брокере (вместо прямых вызовов между компонентами):

- Развязка компонентов – детекторы и модуль слияния не зависят от источника данных и друг от друга, взаимодействуют только через топики;
- Замена источника без модификации обработчиков – реальные сенсоры подключаются заменой адаптера, остальной пайплайн не меняется (это и решает текущую проблему отсутствия физического оборудования: имитатор полностью эмулирует поведение источника);

- Модульность модальностей – режимы *video-only* / *audio-only* / *late* / *hybrid* реализуются естественно (модуль слияния обрабатывает то, что приходит в *inference*);
- Масштабируемость – компоненты можно запускать как независимые процессы, при необходимости – на разных машинах;
- Контролируемость эксперимента – единая точка для внесения искажений («канал деградации») и для сбора метрик (*sink*); воспроизводимость прогонов.

3.2 Программные технологии

Используемые программные технологии и их обоснование сведены в таблицу 2.

Таблица 2 – Программные технологии

Технология	Роль	Обоснование
<i>Python 3.x</i>	основной язык реализации	развитая экосистема <i>ML</i> / <i>CV</i> / обработки аудио; преобладание со стеком ВКР
<i>Apache Kafka (single-broker)</i> [28]	брокер сообщений	поточная обработка, развязка компонентов, замена источника данных; легковесна в <i>single-broker</i> режиме, разворачивается локально / в <i>Docker</i>
<i>kafka-python</i> / <i>confluent-kafka</i>	клиент <i>Kafka</i>	стандартные библиотеки доступа к <i>Kafka</i> из <i>Python</i>
<i>Ultralytics YOLOv8</i> [26], <i>OpenCV</i>	визуальная детекция, обработка изображений	используется в ВКР; <i>YOLOv8</i> – высокая точность на малых объектах при работе в реальном времени; есть отлаженный пайплайн
<i>ByteTrack</i> [27] / <i>Supervision</i>	трекинг объектов, аннотирование/визуализация	используется в ВКР; <i>ByteTrack</i> оптимизирован под детекторы семейства <i>YOLO</i>
<i>PyTorch</i>	<i>DL</i> -фреймворк (акустическая ветка, модуль слияния)	стандартный фреймворк глубокого обучения
<i>librosa</i> [29] / <i>torchaudio</i> / <i>python_speech_features</i>	извлечение аудио-признаков (спектрограммы, <i>MFCC</i>)	стандартные инструменты обработки аудио

Технология	Роль	Обоснование
собственный модуль слияния	агрегация уверенностей, временное выравнивание, адаптивное взвешивание	реализуется в рамках работы
собственный имитатор источников и «канал деградации»	воспроизведение данных в потоке, внесение управляемых искажений	реализуется в рамках работы
<i>Streamlit</i> / <i>PySide6</i> (опц.)	<i>sink</i> -дашборд / визуализация результатов	<i>Streamlit</i> – быстрый веб-дашборд для демонстрации; <i>PySide6</i> – преемственность с интерфейсом ВКР

Сериализация сообщений – *JSON* (на этапе пилотных исследований; при необходимости – *Avro* / *protobuf* со схемой). Топики: *video.raw*, *audio.raw*, *inference*, *decisions*. Кадры в *video.raw* передаются как *JPEG*-байты с пониженным разрешением либо как метаданные со ссылкой на кадр (решается при реализации, исходя из нагрузки на брокер).

3.3 Вычислительные ресурсы и оборудование

Обучение и дообучение моделей – удалённый сервер с *GPU NVIDIA RTX 3090* (24 ГБ *VRAM*): достаточно для обучения/дообучения *YOLOv8* [26], обучения *lightweight CNN* [2], экспериментов с конфигурациями модуля слияния.

Разработка, локальный инференс и замеры производительности – ноутбук с *GPU NVIDIA GTX 1660 Ti* (6 ГБ *VRAM*, *mobile*): рассматривается как модель «целевого» *edge*-устройства – лёгкий инференс на ограниченном железе, замеры *FPS* и латентности именно на такой конфигурации; брокер сообщений *Kafka* разворачивается на этой же машине. Сценарий «обучение на мощной *GPU* → инференс на слабой» отражает реалистичное развёртывание модуля *anti-UAV* и обосновывает требование к лёгкости моделей (раздел 3.2).

Сенсоры: на текущем этапе физические сенсоры не используются – их функция эмулируется имитатором источников данных. Камера предполагается одиночная (на пилотном этапе – воспроизведение кадров и видеороликов из датасетов; опционально, при появлении оборудования, – живой поток с веб-камеры). Собственный микрофон / микрофонный массив – задача на будущее (когда потребуются собственные синхронные аудио-видео записи); на текущем этапе акустическая ветка работает на публичных аудиодатасетах.

3.4 Датасеты

Используемые датасеты и обоснование их выбора приведены в таблице 3.

Таблица 3 – Используемые датасеты

Назначение	Датасеты	Обоснование выбора
Визуальная детекция БПЛА	<i>DUT Anti-UAV</i> [30] ($\approx 10\,000$ размеченных изображений, лицензия MIT) – основной набор для обучения/оценки детектора; <i>Drone-vs-Bird Detection Challenge</i> [31] (содержит негативы «птица» и кадры без дрона); опционально – <i>Anti-UAV300/410 (RGB и ИК, день/ночь)</i> ; собственный комбинированный датасет из ВКР (классы «дрон/птица/самолёт») [36]	публичные, релевантные задаче детекции дрона (а не съёмке с дрона); <i>Drone-vs-Bird</i> критичен для снижения ложных срабатываний на птицах; наборы <i>VisDrone / UAVDT</i> не используются – это съёмка с дрона на наземные объекты, не задача обнаружения дрона
Акустическая детекция/классификация БПЛА	<i>Multiclass Acoustic Dataset</i> [12] (≈ 3200 клипов по 5 с, 32 модели дронов, записи в реальных условиях) – обучение/оценка акустического классификатора; <i>DroneAudioDataset (Al-Emadi)</i> [32] – содержит большой пул негативов «не-дрон»; <i>ESC-50</i> [33] и классы « <i>Propeller / Helicopter / Aircraft engine</i> » из <i>AudioSet</i> [34] – дополнительные источники фоновых звуков для бинарной детекции «дрон / не-дрон»	публичные; <i>Multiclass Acoustic</i> – современный и разнообразный по моделям дронов, записан в реалистичных условиях; <i>Al-Emadi</i> даёт готовое разбиение «дрон / не-дрон»; <i>ESC-50 / AudioSet</i> – стандартные источники окружающих звуков
Пилотный тест слияния (синхронные аудио и видео)	синхронные пары формируются программно адаптером воспроизведения (<i>DatasetReplayAdapter</i>): кадр с дроном модели X сопоставляется с аудиоклипом того же класса; негативная пара – кадр без дрона и фоновый шум; альтернатива –	по-настоящему синхронизированных по времени аудио и видео датасетов БПЛА в открытом доступе фактически нет (кроме <i>MMAUD</i>); формирование «искусственно совмещённых» пар из отдельных модальностей – стандартный приём при

Назначение	Датасеты	Обоснование выбора
	MMAUD (NTU) [35] – единственный готовый публичный синхронный аудио и видео датасет БПЛА (синхронизация модальностей по программным временным отметкам, версии до 100 м, дневные условия)	отсутствии таких данных; такой подход также позволяет точно контролировать межмодальную задержку в экспериментах

3.4.1 Детальное описание классов и баланса датасетов

В таблицу 3 добавлена детализация по классам и балансу, чтобы было понятно, какие метки используются непосредственно в обучении, какие наборы являются источником *hard-negative* примеров и как компенсируется дисбаланс между «дрон» и «не-дрон».

Таблица 4 – дополнение к таблице 3 (классы, баланс и роль датасетов)

Датасет / источник	Классы и метки	Баланс и использование в работе
<i>DUT Anti-UAV</i> [30]	Основной объект детекции – <i>drone</i> / <i>UAV</i> ; фон задаётся областями изображения вне <i>bbox</i> .	Используется для обучения и теста визуальной ветки. Негативный фон внутри кадров дополняется отдельными <i>hard-negative</i> наборами, чтобы снизить <i>false positive</i> .
<i>Drone-vs-Bird</i> [31]	<i>drone</i> , <i>bird</i> , кадры без дрона / <i>background</i> .	Нужен не для увеличения числа <i>positive</i> , а для балансировки ошибок: птицы и пустые кадры выступают <i>hard-negative</i> примерами для визуального детектора.
Комбинированный набор ВКР [36]	<i>drone</i> , <i>bird</i> , <i>airplane</i> .	Используется как предшествующий <i>baseline</i> и как источник классов-помех; при сравнении обязательно считать метрики по классам, а не только общую <i>accuracy</i> .
<i>Multiclass Acoustic Dataset</i> [12]	32 модели дронов; для бинарной постановки все модели сворачиваются в класс <i>drone</i> .	Положительный акустический домен. Так как в наборе доминируют записи БПЛА, для бинарного обучения требуется добавлять <i>non-drone</i> из <i>Al-Emadi</i> / <i>ESC-50</i> / <i>AudioSet</i> .

<i>DroneAudioDataset</i> / <i>Al-Emadi</i> [32]	<i>yes_drone, bebop_1, membo_1</i> и <i>non-drone/background</i> .	В варианте 1 после оконного разбиения использовано около 2.2 тыс. <i>positive drone</i> -окон; дисбаланс компенсируется взвешенной <i>CrossEntropy</i> и/или <i>balanced sampler</i> .
<i>ESC-50</i> [33] и <i>AudioSet</i> [34]	городской шум, транспорт, голоса, <i>aircraft/helicopter/propeller</i> и др. <i>non-drone/hard-negative</i> классы.	Используются как отрицательные и фоновые примеры; это уменьшает риск того, что модель будет считать любой моторный звук дроном.
<i>DADS</i> + <i>DroneAudioSet</i> для <i>AST</i> [37]	Агрегированные <i>drone / non-drone</i> источники; доменные аугментации <i>codec round-trip, urban noise, SpecAugment</i> .	Используются для закрытия <i>domain gap</i> : в пилоте именно <i>AST</i> с предобучением и аугментациями дал 99/144 окон с $p(drone) \geq 0.5$ на целевом аудио.
<i>MMAUD</i> [35]	Синхронные пары <i>audio-video</i> ; сценарии с БПЛА и без БПЛА, дистанции до 100 м.	Основной кандидат для строгой проверки <i>fusion</i> , потому что модальности согласованы по времени, в отличие от искусственно совмещённых пар.

Для всех *train/val/test*-разбиений фиксируется *seed* и отдельно считаются метрики по *positive* и *negative* классам. Это важно, потому что при сильном преобладании фона высокая *accuracy* может скрывать низкий *recall* по классу «дрон».

4 ОПИСАНИЕ ПРОЕКТА СОЗДАВАЕМОГО ПРОДУКТА: ГРАФИЧЕСКИЕ И МАТЕМАТИЧЕСКИЕ МОДЕЛИ

4.1 Графические модели

4.1.1 Общая архитектура системы

Система представляет собой потоковый пайплайн на брокере сообщений *Apache Kafka* [28]: источник данных (имитатор / реальные сенсоры) публикует кадры и аудио-окна в топики; детекторы-консьюмеры (визуальный – *YOLOv8* [26] и *ByteTrack* [27]; акустический – *lightweight CNN* [2]) формируют детекции и публикуют их в общий топик *inference*; модуль слияния агрегирует детекции (позднее / гибридное слияние [4, 9, 10, 11] с адаптивным взвешиванием [17] и временным выравниванием [11]) и публикует решения в топик *decisions*; *sink*-компонент собирает метрики и визуализирует результаты. Опционально между источником и топиками включается «канал деградации». Общая архитектура системы приведена на рисунке 1 в приложении 1.

4.1.2 Диаграмма компонентов

Диаграмма компонентов системы – на рисунке 2 в приложении 1. На диаграмме отражены: набор адаптеров источников (*DatasetReplayAdapter* – кадры из датасетов и аудиоклипы, сшивка в синхронный поток; *VideoFileAdapter* – видеоролик в поток кадров с заданным *FPS*; *LiveSensorAdapter* для веб-камеры и микрофона как направление развития; опциональный *SyntheticAdapter*), модуль «канала деградации» (*DegradationChannel*) с параметрами Δt , p_drop , $audio_noise_snr$, $modality_dropout$, брокер *Apache Kafka* с четырьмя топиками (*video.raw*, *audio.raw*, *inference*, *decisions*), детекторы-консьюмеры (*VisualDetector* – *YOLOv8* и *ByteTrack*; *AcousticDetector* – *lightweight CNN*), модуль слияния (*FusionEngine* – выравнивание и *late/hybrid* и *gating*) и компонент сбора метрик и визуализации (*MetricsSink* / *Dashboard*).

4.1.3 Диаграмма последовательности обработки одного события

Последовательность обработки одного события (кадр и аудио-окно → решение) приведена на рисунке 3 в приложении 1. Источник публикует кадр x_n с временной отметкой t_n в топик *video.raw* и аудио-окно a_m с отметкой τ_m в топик *audio.raw*; визуальный детектор выполняет детекцию *YOLOv8* и ассоциацию *ByteTrack*, акустический – преобразует окно в спектрограмму или *MFCC* и подаёт в *lightweight CNN*; оба публикуют результаты в топик *inference*; модуль слияния потребляет детекции, выполняет выравнивание по окну $[t - \varepsilon, t + \varepsilon]$, применяет позднее или гибридное слияние с *adaptive gating* и публикует итоговое решение в топик *decisions*; *sink*-компонент пишет лог и собирает метрики ($F1$, FPS , $latency$, Δt), визуализирует результаты.

4.1.4 Диаграмма деятельности обработки данных

Диаграмма деятельности приведена на рисунке 4 в приложении 1. В ней показан полный поток обработки от получения синхронизируемых входов до публикации решения. Разветвления оформлены *UML*-ромбами: сначала проверяется доступность модальностей, затем наличие результатов обеих веток, после чего выбирается либо унимодальный путь, либо временное выравнивание и *fusion*. Финальная проверка $p_{fused} \geq \tau$ также показана как *decision node*.

4.1.5 Форматы сообщений в топиках

Форматы сообщений в топиках брокера приведены в таблице 4.

Таблица 4 – Форматы сообщений в топиках *Apache Kafka*

Топик	Поля сообщения	Назначение
<i>video.raw</i>	<i>ts</i> (временная отметка), <i>frame</i> (кадр: <i>JPEG</i> -байты пониженного разрешения либо ссылка), <i>meta</i> (источник, разрешение)	сырой видеопоток
<i>audio.raw</i>	<i>ts</i> , <i>window</i> (аудио-окно: <i>PCM</i> /признаки), <i>meta</i> (источник, <i>sample rate</i> , длина окна)	сырой аудиопоток
<i>inference</i>	<i>ts</i> , <i>modality</i> (<i>video/audio</i>), <i>label</i> (<i>drone/non-drone</i>), <i>confidence</i> $\in [0,1]$, <i>bbox?</i> (для видео), <i>track_id?</i> (для видео), <i>latency</i> (время инференса ветки)	результаты детекторов
<i>decisions</i>	<i>ts</i> , <i>decision</i> (<i>drone/non-drone</i>), <i>confidence</i> (итоговая p_{fused}), <i>contributions</i> (вклады модальностей p_v, p_a, w_v, w_a и поправка (Δ), служебные метрики (Δt , <i>end-to-end latency</i>)	итоговые решения и метрики

4.2 Математические модели

4.2.1 Модель позднего слияния (*late fusion*)

Для согласованного по времени события (см. 5.2.2) пусть $p_v \in [0,1]$ – оценка уверенности визуальной ветки в наличии БПЛА, $p_a \in [0,1]$ – оценка акустической ветки. Итоговая оценка уверенности – взвешенная комбинация:

$$p_{fused} = w_v p_v + w_a p_a, \quad w_v, w_a \geq 0, \quad w_v + w_a = 1. \quad (4.1)$$

Веса w_v, w_a – функции от качества каналов (адаптивное доверие, *gating*; идея адаптивного взвешивания заимствована из подходов с механизмами внимания в акустике [17]): $w_a = g_a(SNR_a)$ растёт с увеличением отношения сигнал/шум аудио; $w_v = g_v(q_v)$ растёт с улучшением качества кадра q_v (резкость, освещённость); после вычисления g_a, g_v веса нормируются: $w_v = g_v/(g_v + g_a)$, $w_a = g_a/(g_v + g_a)$. В простейшем (*baseline*) варианте $w_v = w_a = 1/2$. (4.2)

Правило компенсации ошибок. Базовая оценка p_{fused} корректируется в зависимости от согласованности решений модальностей. Введём корректирующее слагаемое Δ :

$\Delta = +\delta_{conf}$ – если обе модальности активны и обе выдали решение «БПЛА» (взаимное подтверждение – усиление уверенности, поддержка *true positive*); (4.3а)

$\Delta = -\delta_{unconf}$ – если обе модальности активны, но решение «БПЛА» выдала только одна из них (неподтверждённая детекция – понижение уверенности, подавление возможного *false positive*); (4.3б)

$\Delta = 0$ – в остальных случаях, в том числе, когда вторая модальность отсутствует или отказала (тогда система работает в унимодальном режиме, штраф не применяется). (4.3в)

Скорректированная оценка уверенности: (4.4)

$$\tilde{p}_{fused} = \min(1, \max(0, p_{fused} + \Delta)), (4.4)$$

где $\delta_{conf}, \delta_{unconf} \geq 0$ – малые параметры (бонус и штраф), подбираемые по валидационной выборке; операция $\min(1, \max(0, \dots))$ ограничивает результат интервалом $[0,1]$.

Решающее правило: $y = \text{«БПЛА»}$, если $\tilde{p}_{fused} \geq \tau$, иначе $y = \text{«не-БПЛА»}$; порог τ подбирается по валидационной выборке (компромисс *precision/recall*). В режимах *video-only* / *audio-only* итоговая оценка равна уверенности соответствующей активной ветки (т.е. $p_{fused} = p_v$ или $p_{fused} = p_a$, $\Delta = 0$). (4.5)

Обоснование формулы p_{fused} . Формула (4.1) рассматривается как интерпретируемый *baseline* позднего слияния на уровне решений, то есть как смесь двух экспертов: визуального и акустического. После калибровки вероятностей на *validation*-сплите оценки p_v и p_a можно агрегировать выпуклой комбинацией, а веса w_v и w_a отражают доверие к каналам. Поэтому модель не претендует на «абсолютную» физическую вероятность, а задаёт воспроизводимое правило ранжирования событий по уверенности.

Чтобы уменьшить эмпиричность Δ -поправки (4.3), параметры δ_{conf} , δ_{unconf} и порог τ подбираются только на *validation*-выборке по сетке с критерием максимизации $F1$ при ограничении на *false positive* и FPS . На *test*-сплите эти параметры не меняются. В отчёт также добавлено *ablation*-сравнение с ошибающейся и исправленной акустической веткой, что показывает границу применимости правила: при рассогласованных модальностях *fusion* не должен автоматически считаться лучше *video-only*.

Практически это означает следующий протокол: сначала проверяется *baseline* без поправки $\Delta = 0$, затем вводится симметричная/асимметричная поправка (δ_{conf} , δ_{unconf}), после чего сравниваются *precision*, *recall*, $F1$ и доля ложных срабатываний. Такая процедура

делает формулу p_{fused} не произвольной эвристикой, а калибруемым *decision-level fusion* правилом.

Гибридное слияние (направление развития) дополняет схему агрегацией не только финальных уверенностей, но и промежуточных признаков веток (*feature-level* и *decision-level* [10, 11]); решающее правило сохраняется.

4.2.2 Модель временного выравнивания (*alignment*)

Видеопоток – кадры с частотой f_v (интервал между кадрами $1/f_v$); аудиопоток – окна длины T_a с шагом h_a . Детекция визуальной ветки в кадре с временной отметкой t_N сопоставляется с детекцией акустической ветки по аудио-окну a_m , если интервал окна $[\tau_m, \tau_m + T_a]$ перекрывается с интервалом $[t_N - \varepsilon, t_N + \varepsilon]$, где ε – допуск выравнивания (параметр системы). Если для визуальной детекции не находится сопоставимого аудио-окна (или наоборот), событие обрабатывается в унимодальном режиме. (4.6)

Межмодальная задержка $\Delta t = |t_N - \tau_m^*|$ для сопоставленной пары (где τ_m^* – центр сопоставленного аудио-окна). Исследуется зависимость метрик качества (*precision*, *recall*, *F1*) от Δt : ожидается деградация качества слияния при росте Δt сверх некоторого порога; задача – определить допустимый диапазон Δt и устойчивость к нему. (4.7)

4.2.3 Модель «канала деградации»

Для воспроизводимого исследования устойчивости системы вводится модель параметризуемых искажений входного потока:

- Δt_{shift} – управляемый сдвиг аудиопотока относительно видеопотока (имитация рассинхронизации сенсоров);
- $p_{dropout} \in [0,1]$ – вероятность дропа отдельного кадра (имитация потерь в канале);
- SNR_{target} – целевое отношение сигнал/шум при добавлении шума в аудио (имитация шумной обстановки);
- $T_{dropout}$ – интервалы полного отключения одной из модальностей (имитация отказа сенсора, *missing modality* [25]).

Эти параметры задают условия стресс-сценариев (раздел 5.2): прогоны системы при различных значениях параметров $\rightarrow$ построение кривых «метрика vs величина искажения». Сценарии Δt_{shift} и $T_{dropout}$ направлены на воспроизводимую проверку пробелов литературы – отсутствие явного исследования межмодальной задержки [11] и устойчивости к отсутствующей модальности [25] в прикладных *anti-UAV* работах (раздел 1.2).

4.2.4 Метрики

- $Precision = TP/(TP + FP)$, $Recall = TP/(TP + FN)$, $F1 = 2 \cdot precision \cdot recall / (precision + recall)$, где TP, FP, FN – число истинно-положительных, ложно-положительных и ложно-отрицательных решений. (4.8)

- $mAP@IoU$ (для визуальной ветки) – средняя точность по классам при заданном пороге IoU между предсказанной и истинной рамками.

- FPS – число обработанных кадров в секунду.

- $End-to-end\ latency$ – время от поступления кадра / аудио-окна в систему до публикации соответствующего решения в $decisions$ (включая время прохождения через брокера сообщений и время инференса).

- Ресурсы: число параметров модели, $FLOPs$ (число операций с плавающей точкой на одно предсказание), пиковое потребление памяти.

- Прирост от слияния: $\Delta F1 = F1_{fusion} - \max(F1_{video-only}, F1_{audio-only})$ – ключевая величина (целевое значение – раздел 5.2). (4.9)

5 ПЛАН ИСПЫТАНИЙ ПРОДУКТА И ПЕРЕЧЕНЬ ОЖИДАЕМЫХ РЕЗУЛЬТАТОВ

5.1 Тестовый стенд

Тестовый стенд – это сам потоковый пайплайн в режиме воспроизведения: имитатор источников (*DatasetReplayAdapter* / *VideoFileAdapter*, данные – из [12, 30, 31, 32, 33]) → *Apache Kafka* [28] → детекторы-консьюмеры (*YOLOv8* [26] и *ByteTrack* [27]; *lightweight CNN* [2]) → модуль слияния → *sink* с автоматическим сбором метрик. Режимы (*video-only* / *audio-only* / *late* / *hybrid*) переключаются конфигурацией стенда. Опционально в тракт включается «канал деградации» (модель 4.2.3) для стресс-сценариев. Все прогоны воспроизводимы (фиксируются версии моделей, сплиты данных, параметры стенда).

5.2 План экспериментов

Данные и разбиения: визуальная ветка – *DUT Anti-UAV* [30] и *Drone-vs-Bird* [31], разбиение *train/val/test*; акустическая – *Multiclass Acoustic* [12] и *Al-Emadi* [32] и *ESC-50* [33] (негативы), разбиение *train/val/test*; для слияния – синхронные пары через *DatasetReplayAdapter* (либо *MMAUD* [35]) с фиксированным сценарием совмещения.

Метрики (определения – раздел 4.2.4): *precision* / *recall* / *F1*, *mAP@IoU* (визуальная ветка), *FPS*, *end-to-end latency*, ресурсы, межмодальная задержка Δt .

Базовый сценарий: «тепличные» идеально синхронные данные (без искажений) → *ablation*-сравнение четырёх режимов *video-only* / *audio-only* / *late fusion* / *hybrid fusion*; ключевая величина – прирост $\Delta F1$ от слияния относительно лучшей унимодальной ветки.

Стресс-сценарии (через «канал деградации»): прогоны при варьируемых параметрах искажений → кривые «метрика vs величина искажения» – (а) сдвиг Δt_{shift} аудио ↔ видео от 0 до заданного максимума; (б) дроп кадров p_{drop} от 0 до заданного максимума; (в) понижение *SNR* аудио (шумная обстановка); (г) отключение одной из модальностей ($T_{dropout}$ – проверка устойчивости к *missing modality*); (д) ухудшение визуального канала (сложный фон, низкая освещённость). Эти сценарии напрямую проверяют «белые пятна» 1, 2 и 4 из раздела 1.2 (временное выравнивание, шумные/реалистичные условия, устойчивость к отсутствующей модальности).

Целевые значения (критерии достижения цели): прирост $\Delta F1$ на 10–15 % при *FPS* ≥ 20 ; снижение *false positive* / *false negative* за счёт компенсации ошибок между каналами; контролируемая (предсказуемая) межмодальная задержка Δt ; плавная (без резкого «обвала») деградация качества при отключении одной модальности.

5.3 Перечень ожидаемых результатов применения на практике

Система мультимодального обнаружения БПЛА может применяться для: обеспечения безопасности критически важных объектов инфраструктуры (АЭС, НПЗ, объекты энергетики, режимные территории) – своевременное обнаружение несанкционированных БПЛА; контроля воздушного пространства над массовыми мероприятиями (спортивные соревнования, концерты, общественные собрания); интеграции в системы управления воздушным движением для совместного безопасного использования воздушного пространства; научных исследований поведения БПЛА. Ожидаемый выход системы при практическом применении – поток решений «обнаружен БПЛА / не обнаружен» с оценкой уверенности, временной отметкой и (для визуального канала) локализацией объекта и идентификатором трека. Модульность (поддержка режимов *video-only* / *audio-only* / *late* / *hybrid*) позволяет применять одну модальность или их комбинацию в зависимости от доступного оборудования и условий эксплуатации; адаптивное взвешивание модальностей повышает надёжность в изменяющихся условиях; устойчивость к отказу одной из модальностей – важное свойство для эксплуатации в реальных условиях.

5.4 Пробные замеры

Проведены первые прогоны базового сценария на тестовом стенде – для проверки корректности методики измерений, работоспособности инструментов (развёртывание *Kafka* [28], адаптеры источников, детекторы-консьюмеры на *YOLOv8* [26] и *lightweight CNN* [2], *sink*-метрики) и получения первых ориентировочных значений (доля решений «дрон», средняя p_{fused} , FPS , $end-to-end\ latency$). По итогам пробных замеров обнаружен и устранён ряд дефектов реализации (перенормировка весов *fusion* при одном активном канале; сопоставление детекций с треками по *IoU* вместо индекса; приоритет *drone*-детекций при выборе лучшей детекции в окне), а также подобраны параметры выравнивания ($window_epsilon_ms = 600$ – соизмерим с шагом аудио-окна). Содержательные результаты, включая *ablation*-сравнение режимов, – раздел 6.

6 ПИЛОТНЫЕ ИССЛЕДОВАНИЯ: РЕАЛИЗАЦИЯ И РЕЗУЛЬТАТЫ

6.1 Реализация потокового пайплайна

В соответствии с архитектурой, заданной в разделе 3, реализована система из семи микросервисов, взаимодействующих через брокера сообщений *Apache Kafka* [28]:

- *source-simulator* – имитатор источников; адаптер *MediaFileAdapter* стримит синхронные кадры видео + окна аудио из «тепличного» набора *sandbox-video/audio-for-simulator*; есть также *DatasetReplayAdapter* для воспроизведения из публичных датасетов [30, 31];
- *ingest-gateway* – нормализация и публикация сообщений в топики *video.raw* / *audio.raw*;
- *visual-detector* – *YOLOv8* [26] + *ByteTrack* [27]; дообученные веса под класс «БПЛА» – раздел 6.2;
- *acoustic-detector* – *lightweight CNN* [2] по *mel*-спектрограммам / *MFCC*; сменный *backend* – раздел 6.2;
- *fusion* – временное выравнивание [11], *late* / *hybrid* стратегии, адаптивное взвешивание [17];
- *sink* – запись *inference* / *decisions* в *PostgreSQL*, *JSONL*-журнал, метрики *Prometheus* :9108;
- *gateway* – дашборд *REST/WS* на основе данных из *decisions*.

Инфраструктура – *Kafka KRaft (single-broker)*, *PostgreSQL 16* со схемой *uavdet* и миграциями *Liquibase*, оркестрация через *Docker Compose*; запуск пилотных прогонов воспроизводится одной командой. Полная декомпозиция и диаграммы реализованной системы – раздел 4.1.

6.2 Обученные/подключённые модели детекторов

Визуальный детектор. В рамках исследования был обучен *YOLOv8n* под одну предметную задачу – детекция класса «дрон»: дообучение от предобученных весов на наборе *DroneDetectionDataset* [HF-зеркало *pathikg/drone-detection-dataset*, ~ 54 тыс. кадров, лицензия *MIT*], *imgsz* = 480, 20 эпох, обучение на *NVIDIA RTX 3090*. По итогам на *test*-сплите получены: $mAP@0.5 = 0.881$, $precision = 0.906$, $recall = 0.813$, $mAP@0.5:0.95 = 0.430$ (*val*-метрики на 20-й эпохе: $P = 0.988$, $R = 0.970$, $mAP@0.5 = 0.986$, $mAP@0.5:0.95 = 0.762$). Этот результат используется как основа визуальной ветки во всех пилотных прогонах.

Акустический детектор. В ходе пилота акустическая ветка прошла через три последовательных варианта, отражающих процесс закрытия *domain gap* (см. далее 6.4). Сравнение вариантов приведено в таблице 5.

Таблица 5 – Три варианта акустического детектора

Параметр	Вариант 1: <i>lwcnn</i> / <i>DroneAudioData set</i> / <i>MFCC</i>	Вариант 2: <i>lwcnn</i> / <i>DADS</i> / <i>mel</i> - спектрограм мы	Вариант 3: <i>AST</i> [37] (<i>samid-drone-detector</i>)
Архитектура	<i>lightweight CNN</i> [2]	<i>lightweight CNN</i> [2] (та же, что в варианте 1)	<i>Audio Spectrogram Transformer (AST-base)</i> [37]
Источник весов	обучен с нуля	обучен с нуля	дообучение <i>AST-base</i> , предобученного на <i>AudioSet</i> [34] (~ 2 млн природных и техногенных звуков)
Обучающие данные	положительные классы <i>yes_drone</i> + <i>bebop_1</i> + <i>membo_1</i> из <i>DroneAudioData set (Al-Emadi)</i> [32] (≈ 2.2 тыс. дрон-окон); негативы – <i>ESC-50</i> [33] (≈ 18 тыс. фон-окон)	<i>DADS</i> – агрегат 6 источников звука дронов и 4 не-дрон, ≈ 180 тыс. клипов	<i>DADS</i> + <i>DroneAudioSet</i> с аугментациями: <i>codec round-trip</i> , <i>urban-noise overlay</i> , <i>SpecAugment</i> [38], <i>Mixup</i>
Входные признаки	<i>MFCC</i> 40×64	<i>mel</i> -спектрограммы 64×64	128-мел-спектрограмма (собственный <i>ASTFeatureExtractor: PCM $\rightarrow$ mel</i>)
Параметры окна	1 с с шагом 0.5 с	0.5 с с шагом 0.25 с, балансировка по числу окон на файл	стандартные окна <i>AST</i>
Функция потерь / отбор	взвешенный <i>CrossEntropy</i> , выбор лучшего чекпойнта по сбалансированной точности	то же, что в варианте 1	стандартный <i>fine-tune AST</i>
<i>Test</i> -метрики на «собственно м» домене	<i>accuracy</i> = 0.993, <i>F1_macro</i> = 0.980, <i>recall(drone)</i> $\approx$ 0.986, <i>recall(non-drone)</i> $\approx$ 0.992	<i>accuracy</i> = 0.995, <i>F1_macro</i> = 0.995	характеристика модели – <i>backbone</i> предобучен на <i>AudioSet</i> (~ 2 млн звуков)

Параметр	Вариант 1: <i>lwcnn / DroneAudioDataSet / MFCC</i>	Вариант 2: <i>lwcnn / DADS / mel-спектрограммы</i>	Вариант 3: <i>AST</i> [37] (<i>samid-drone-detector</i>)
Интеграция в <i>acoustic-detector</i>	первый <i>backend</i>	второй <i>backend</i>	третий <i>backend</i> (<i>arch = ast</i>); инференс через <i>transformers.AutoModelForAudioClassification</i>
Поведение на целевом аудио (см. 6.4)	формально близко к идеалу на тестовом сплите, но на целевом аудио систематически ошибается (см. 6.4)	ещё точнее на «собственно м» домене, но <i>domain gap</i> не закрыт (см. 6.4)	реально работает на целевом аудио – раздел 6.4

Выбор архитектурного решения для акустической ветки (*lightweight CNN*, раздел 2.2.2) при этом сохраняется: *AST* реализован как опциональный сменный *backend*, а не как смена парадигмы.

Графики по основным метрикам дообучения вынесены в приложение 1: рисунок 5 показывает контрольные значения *precision*, *recall*, *mAP@0.5* и *mAP@0.5:0.95* для *YOLOv8n*; рисунок 6 в приложении 1 показывает сравнение вариантов акустического детектора. В дальнейшем при наличии полного файла *results.csv* эти рисунки заменяются на *epoch-wise curves loss/precision/recall/mAP* без изменения структуры отчёта.

6.3 Прогоны базового сценария

Для проверки работоспособности всего пайплайна и сбора первых ориентировочных значений были выполнены два сквозных прогона на одном и том же «тепличном» наборе данных (синхронные видео и аудио, длительность 72.5 с, 25 кадров/с, аудио ресемплируется к 16 кГц моно) на двух разных стендах. Архитектура реализации одинакова, различие — только в платформе исполнения визуальной ветки (*CPU x86_64 vs GPU CUDA*). Характеристики стендов приведены в таблице 6, сводное сравнение прогонов — в таблице 7.

Таблица 6 – Характеристики стендов

Параметр	Стенд 1	Стенд 2
Класс устройства	ноутбук	настольный компьютер

Параметр	Стенд 1	Стенд 2
Процессорная архитектура	<i>ARM64 (Apple Silicon, MacBook)</i>	<i>x86_64</i>
Платформа визуальной ветки	<i>CPU, Docker-сборка по умолчанию</i>	<i>GPU CUDA, Dockerfile.cuda</i>
<i>GPU</i>	не используется	<i>NVIDIA GTX 1660 Ti, 6 ГБ VRAM (mobile)</i>
Назначение	пилотный запуск, проверка работоспособности пайплайна на ограниченных ресурсах	пилотный запуск для замеров производительности и латентности в режиме реального времени

Таблица 7 – Сравнение прогонов базового сценария (*CPU vs GPU*)

Параметр	Прогон № 1 (<i>ARM64, CPU</i>)	Прогон № 2 (<i>x86_64, GPU NVIDIA GTX 1660 Ti, 6 ГБ</i>)
Платформа визуальной ветки	<i>CPU, Docker-сборка по умолчанию</i>	<i>GPU CUDA, Dockerfile.cuda</i>
Режим слияния	<i>late fusion</i>	<i>late fusion</i>
Акустический backend	<i>lwcnn-MFCC</i> (вариант 1, см. 6.2)	<i>lwcnn-MFCC</i> (вариант 1, см. 6.2)
Латентность визуальной ветки	$\approx 185\text{--}210$ мс/кадр	$\approx 9\text{--}13$ мс/кадр (ускорение $\approx \times 20$)
<i>End-to-end</i> задержка решений	растёт до сотен секунд (накапливается)	≈ 0.7 с, не накапливается
Удержание реального темпа <i>25 fps</i>	не удерживается даже при <i>5 fps</i>	удерживается с запасом
Целевой <i>FPS</i> ≥ 20 (раздел 5.2)	не достигнут	подтверждён
Объём накопленных данных	≈ 1300 строк <i>inference</i> и ≈ 1300 <i>decisions</i> за ~ 5 мин	сопоставимый объём, темп публикации соответствует темпу источника
Основной результат прогона	пайплайн работает целиком (<i>Kafka</i> $\rightarrow$ детекторы $\rightarrow$ <i>fusion</i> $\rightarrow$ <i>PostgreSQL</i> $\rightarrow$ дашборд); устранён дефект перенормировки весов <i>late/hybrid</i> при одном активном канале	выявлено качественное расхождение: акустика <i>lwcnn-MFCC</i> классифицирует все окна целевого аудио как <i>non-drone</i> – систематическая ошибка, обусловленная <i>domain gap</i> (см. 6.4)

Дополнительный результат — архитектурная независимость реализации. Все семь микросервисов (раздел 6.1) собираются и запускаются без изменений как на *ARM64* (Стенд 1, *Apple Silicon*), так и на *x86_64* (Стенд 2): используются официальные мультиархитектурные образы *Apache Kafka* и *PostgreSQL*, а собственный код на *Python 3.11* архитектурно-независим. Это снижает требования к целевому оборудованию при практическом развёртывании системы.

6.4 Заккрытие *domain gap* акустической ветки

Расхождение, обнаруженное в прогоне № 2, повлияло на форму *ablation*-эксперимента (раздел 6.5) и привело к отдельному исследованию: насколько обученная на «чистом» лабораторном аудио модель применима к звуку, извлечённому из видеозаписи. Целевой набор — *sandbox-audio-for-simulator.wav*: реальная аудиодорожка видеоклипа, прошедшая видеокодек, с фоном и низким *SNR* (грубая оценка $SNR \approx 2$ дБ). Для каждого варианта акустического детектора (см. 6.2) проведена диагностика: распределение апостериорной вероятности $p(drone)$ по окнам в 1 с (для *AST*) или 0.5 с (для *lwcnn-DADS*) — через инструмент *scripts/probe_audio.py*, прогоняющий *wav* через рантайм-код сервиса. Результаты приведены в таблице 8.

Таблица 8 — Сравнение вариантов акустического детектора на целевом аудио

Вариант акустического детектора	<i>Test</i> -метрики на «своём» домене	На целевом аудио (<i>sandbox</i>)
(1) <i>lwcnn</i> / <i>DroneAudioDataset</i> / <i>MFCC</i>	$acc = 0.993, F1_macro = 0.980$	100 % окон классифицированы как <i>non-drone</i>
(2) <i>lwcnn</i> / <i>DADS</i> / <i>mel</i> -спектр (180 тыс. клипов)	$acc = 0.995, F1_macro = 0.995$	$p(drone) \leq 0.026$ на всех окнах
(3) <i>AST</i> [37] <i>samid-drone-detector</i> (бэкбон <i>AudioSet</i> [34] и аугментации, в т. ч. <i>SpecAugment</i> [38])	характеристика модели — <i>backbone</i> предобучен на <i>AudioSet</i> (~ 2 млн звуков)	99 из 144 окон с $p(drone) \geq 0.5$ ($\approx 69\%$), медиана = 0.632, $p90 = 0.798$

Содержательный вывод. Чисто количественное наращивание «своего» обучающего датасета (от 20 тыс. окон *DroneAudioDataset/ESC-50* до 180 тыс. клипов *DADS*) не закрывает *domain gap* между чистыми микрофонными записями и звуком, прошедшим через видеокодек. Закрывает *gap* именно (а) *backbone*, предобученный на больших разнородных аудио-корпусах (*AudioSet* [34] — естественная робастность к фоновым сценам), и (б) аугментации на этапе *fine-tune*, имитирующие целевой канал (*codec round-trip*, *urban-noise overlay*, *SpecAugment* [38]). Это согласуется с современной литературой по *domain adaptation* в аудио-детекции и независимо подтверждено на нашей задаче.

6.5 Ablation-сравнение режимов слияния

В соответствии с планом испытаний (раздел 5.2) выполнено **ablation-сравнение** **четырёх режимов** работы системы (плюс пятый – *late* и *adaptive gating*) на одном и том же тестовом наборе, на *GPU GTX 1660 Ti*, по 90 с накопления на каждый режим. Метрики снимались из таблиц *PostgreSQL uavdet.inference / uavdet.decisions*. *Ablation* проводился **дважды** – до и после закрытия *domain gap* акустики (см. 6.4) – чтобы зафиксировать вклад слияния в двух качественно разных ситуациях. Результаты *ablation* № 1 представлены в таблице 9, результаты *ablation* № 2 – в таблице 10.

Таблица 9 – *Ablation* № 1 (акустический *backend* = *lwcnn-MFCC*; акустическая ветка систематически ошибается на целевом аудио)

Режим	Доля решений «дрон»	Средняя p_{fused}	Комментарий
<i>video-only</i>	1.000	0.778	визуальный детектор уверенно видит БПЛА
<i>audio-only</i>	0.000	0.000	акустика <i>systematic-FN</i> на целевом звуке (см. 6.4)
<i>late fusion</i> ($w_v = w_a = 0.5$)	0.246	0.404	хуже <i>video-only</i> – наивное усреднение «топит» уверенную видео-детекцию
<i>hybrid fusion</i>	0.231	0.361	дополнительный штраф δ_{unconf} за противоречие каналов
<i>late</i> и <i>adaptive gating</i>	0.261	0.417	<i>gating</i> по <i>SNR</i> почти не помог – звук «чистый», ошибается классификатор

Таблица 10 – *Ablation* № 2 (акустический *backend* = *AST*; акустическая ветка работает на целевом аудио)

Режим	Доля решений «дрон»	Средняя p_{fused}	Окон с активным аудио (<i>TRUE</i>)	Комментарий
<i>video-only</i>	1.000	0.774	–	базовый случай: видео – основной источник решения
<i>audio-only</i>	0.696	0.486	102 из 147	<i>AST</i> реально работает, совпадает с <i>probe</i> ($\approx 69\%$)

Режим	Доля решений «дрон»	Средняя p_{fused}	Окон с активным аудио ($TRUE$)	Комментарий
<i>late fusion</i> ($w_v = w_a = 0.5$)	0.969	0.732	102	согласованное мультимодальное решение
<i>hybrid fusion</i>	0.976	0.736	102	чуть выше <i>late</i> благодаря более жёсткому δ_{unconf}
<i>late</i> и <i>adaptive gating</i>	0.977	0.742	102	максимум; <i>gating</i> ослабляет вес аудио при низком SNR , видео уверенно принимает решение

Системные показатели (общее по *ablation* № 2, GPU). Латентность визуальной ветки ≈ 12 мс/кадр (*CUDA*), акустической (*AST*) ≈ 146 мс/окно (*overhead* – построение мел-спектрограммы средствами *transformers* на *CPU*, при шаге аудио-окна 500 мс укладывается с большим запасом); *end-to-end*-задержка свежих решений в *video-only* ≈ 0.85 с; в *late/hybrid* накапливается из-за узкого места в *sink/PostgreSQL* – отдельный пункт в плане доработок (6.6).

6.6 Интерпретация и корректировка плана

Сопоставление с целевыми значениями (раздел 5.2):

- Производительность $FPS \geq 20$ – подтверждено: визуальная ветка ≈ 12 мс/кадр на *GPU* (запас по реальному времени для 25 *fps* есть).
- Прирост точности от слияния относительно лучшего унимодального решения – в *ablation* № 1 (систематически ошибающаяся акустика) слияние ухудшает решение (доля «дрон» падает с 1.0 до 0.23–0.26). Это содержательный результат, подтверждающий корректность Δ -правила компенсации (раздел 4.2.1) и очерчивающий границу применимости *fusion*: выигрыш достигается только на согласованных модальностях. После закрытия *domain gap* (*ablation* № 2) слияние работает как заявлено: *late* и *adaptive gating* даёт 0.977 при *video-only* = 1.000 с заведомым гонением *TRUE* даже на пустых сегментах; на разметке по секундам выигрыш будет более явным.
- Контролируемая межмодальная задержка Δt – реализована через параметр *window_epsilon_ms* модели временного выравнивания (4.2.2); подобрано $\varepsilon = 600$ мс (соизмеримо с шагом аудио-окна 500 мс) – мультимодальные окна реально формируются.

- *Adaptive gating* – направление подтверждено количественно (и 0.8 п. п. к *late*), но прирост невелик: *gating* на основе *SNR* на входе отвечает на вопрос «деградирован ли канал», а нужно – «надёжно ли решение детектора»; рассматривается доработка – *gating* по энтропии распределения вероятностей на выходе детектора.

Связь с «белыми пятнами» (раздел 1.2):

- Пробел 1 («временное выравнивание»): введена явная модель времени с измеримым параметром ε , окна выравнивания валидируются по доле мультимодальных событий.

- Пробел 2 («реалистичные шумовые условия»): фактически встречен на этапе пилота в виде *domain gap* акустики; закрыт через подключение *AST* с предобучением на *AudioSet* и доменными аугментациями (раздел 6.4).

- Пробел 3 («метрики вычислительных ресурсов»): зафиксированы латентности обеих веток на *GPU* и *CPU*.

- Пробел 4 («устойчивость к отсутствующей модальности»): реализован штатно – режимы *video-only* / *audio-only* запускаются как валидные конфигурации; перенормировка весов *fusion* при одном активном канале выполнена (прогон № 1).

Корректировка плана исследований. На остаток работы и на магистерскую диссертацию вынесены:

- *gating* по энтропии распределения вероятностей на выходе детектора (вместо/в дополнение к *gating* по *SNR*);

- наземная истина (*ground truth*) на «тепличном» наборе по секундам – для строгой оценки $\Delta F1$ от слияния через *precision* / *recall* / *F1*, а не через косвенную долю «дрон»;

- стресс-сценарии через «канал деградации» (раздел 4.2.3) – теперь, когда обе ветки реально работают, эксперименты по Δt / p_{drop} / *SNR* / отключению модальности имеют содержательный смысл;

- переход к синхронному датасету *MMAUD* [35] – содержательные *fusion*-метрики на по-настоящему согласованных модальностях (адаптер *mmaud_replay* подготовлен);

- профилирование *sink/PostgreSQL* – устранение накопления *e2e*-задержки в *late/hybrid*;

- (опционально) дообучение *ResNet-18* с аугментациями как сравнительный вариант акустического *backend'a* к *AST* – для отчётной таблицы.

ЗАКЛЮЧЕНИЕ

В рамках НИР выполнено: сформулирована постановка задачи бинарного обнаружения БПЛА в потоке разнородных данных; определены модели и методы решения (визуальная ветка — *YOLOv8* [26] и *ByteTrack* [27]; акустическая — *lightweight CNN* по спектрограммам/*MFCC* [2] с возможностью смены *backend*; стратегия слияния — позднее слияние на уровне уверенностей [4, 9, 10] с адаптивным взвешиванием [17] и явным временным выравниванием [11], с заделом на гибридное слияние [10, 11]); обоснован выбор технологий и оборудования (поточная архитектура на брокере сообщений *Apache Kafka* [28] с имитатором источников данных, инвариантная к источнику; вычислительные ресурсы; публичные датасеты [12, 30, 31, 32]); разработаны графические и математические модели создаваемого продукта; сформирован план испытаний и перечень ожидаемых результатов применения, проведены пробные замеры на тестовом стенде (раздел 5.4); реализован сквозной пайплайн из семи микросервисов и выполнены пилотные исследования — обучен визуальный детектор *YOLOv8*, обучены и подключены три варианта акустического детектора, проведено двойное *ablation*-сравнение режимов слияния (раздел 6.5), по итогам интерпретации результатов выполнена корректировка плана исследований на остаток магистратуры (раздел 6.6); подготовлен настоящий отчёт о НИР по ГОСТ 7.32-2017 и черновик статьи в журнал РИНЦ.

Полученные результаты позволяют сделать **два содержательных вывода**, которые войдут в задел магистерской диссертации:

1. Мультимодальное слияние работает в направлении, заданном целью НИР, но только на согласованных модальностях. При систематической ошибке одной из веток (*ablation* № 1) наивное *late/hybrid* слияние ухудшает решение относительно унимодального; это подтверждает корректность Δ -правила компенсации и одновременно очерчивает границу применимости *fusion*. После закрытия *domain gap* акустики (раздел 6.4) слияние сходится к согласованной оценке (доля «дрон» 0.969–0.977 в *late/hybrid/gating* против 0.696 у *audio-only* и 1.000 у *video-only*). Требование $FPS \geq 20$ на *GPU* подтверждено (≈ 12 мс/кадр у визуальной ветки).

2. *Domain gap* в акустической ветке закрывается не размером собственного «чистого» датасета, а предобучением на крупном разнородном корпусе (*AudioSet* [34]) и аугментациями целевого канала (*codec round-trip*, *urban-noise overlay*, *SpecAugment*). Эта закономерность согласуется с современной литературой по *domain adaptation* в аудио-детекции и независимо подтверждена на нашем наборе.

Подход с имитатором источников и потоковой архитектурой позволил провести пилотные исследования в условиях, приближённых к реальным, при отсутствии

физического сенсорного оборудования, и обеспечивает воспроизводимое исследование устойчивости системы (через «канал деградации»), что закрывает ряд пробелов, выявленных в аналитическом обзоре [11, 12, 25].

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Alla I., Yahia S., Loscri V. *TRIDENT: Tri-modal Real-time Intrusion Detection Engine for New Targets* // *arXiv preprint*. – 2025. – *arXiv*:2504.06417. – Текст : электронный. – URL: <https://arxiv.org/abs/2504.06417> (дата обращения: 01.12.2025).
2. Aydin I., Emrullah K., *Development of a new Light-Weight Convolutional Neural Network for acoustic-based amateur drone detection* // *Applied Acoustics*. – 2022. – Т. 193. – С. 108773. – Текст : электронный. – URL: <https://www.sciencedirect.com/science/article/pii/S0003682X22001475> (дата обращения: 01.12.2025).
3. Cao S., Wang T., Li T., Mao Z., *UAV small target detection algorithm based on an improved YOLOv5s model* // *Journal of Visual Communication and Image Representation*. – 2023. – Текст : электронный. – URL: <https://www.sciencedirect.com/science/article/pii/S1047320323001864> (дата обращения: 01.12.2025).
4. Dafrallah S., Akhloufi M., *Malicious UAV detection using various modalities* // *Defence Technology*. – 2024. – Текст : электронный. – URL: <https://www.sciencedirect.com/org/science/article/pii/S2564493924000134> (дата обращения: 01.12.2025).
5. Deng T., Zhou Y., Wu W., et. al. *Multi-Modal UAV Detection, Classification and Tracking Algorithm – Technical Report for CVPR 2024 UG2 Challenge* // *arXiv preprint*. – 2024. – *arXiv*:2405.16464. – Текст : электронный. – URL: <https://arxiv.org/abs/2405.16464> (дата обращения: 01.12.2025).
6. Di J., Feng F., Yang Y., Zhang W., *UAV image object detection based on improved YOLOv5s* // *International Conference on Algorithms, High Performance Computing, and Artificial Intelligence (AHPCAI 2023)*. – *SPIE*, 2023. – Т. 12941. – С. 892–897. – Текст : электронный. – URL: <https://www.spiedigitallibrary.org/conference-proceedings-of-spie/12941/129413J/UAV-image-object-detection-based-on-improved-YOLOv5s/10.1117/12.3011970.short> (дата обращения: 01.12.2025).
7. Dumitrescu C., Minea M., Costea I. M., Cosmin Chiva I., Semenescu A. *Development of an Acoustic System for UAV Detection* // *Sensors*. – 2020. – Т. 20. – №. 17. – С. 4870. – Текст : электронный. – URL: <https://www.mdpi.com/1424-8220/20/17/4870> (дата обращения: 01.12.2025).
8. Feng Y., Wang T., Jiang Q., Zhang C., Sun S., Qian W. *A Efficient and Accurate UAV Detection Method Based on YOLOv5s* // *Applied Sciences*. – 2024. – Т. 14, № 15. – С. 6398.

– Текст : электронный. – URL: <https://www.mdpi.com/2076-3417/14/15/6398> (дата обращения: 01.12.2025).

9. Jamil S., Rahman M., Ullah A., et. al. *Malicious UAV Detection Using Integrated Audio and Visual Features for Public Safety Applications* // *Sensors*. – 2020. – Т. 20. – №. 14. – С. 3923. – Текст : электронный. – URL: <https://www.mdpi.com/1424-8220/20/14/3923> (дата обращения: 01.12.2025).

10. Jiao T., Guo C., Feng X., et. al. *A Comprehensive Survey on Deep Learning Multi-Modal Fusion: Methods, Technologies and Applications* // *Computers, Materials & Continua*. – 2024. – Т. 80. – №. 1. – Текст : электронный. – URL: <https://www.techscience.com/cmcl/v80n1/57427> (дата обращения: 01.12.2025).

11. Li S., Tang H. *Multimodal Alignment and Fusion: A Survey* // *arXiv preprint*. – 2024. – arXiv:2411.17040. – Текст : электронный. – URL: <https://arxiv.org/abs/2411.17040> (дата обращения: 01.12.2025).

12. Linn M., Wang Y., Andrew P., et. al. *A Multiclass Acoustic Dataset and Interactive Tool for Analyzing Drone Signatures in Real-World Environments* // *arXiv preprint*. – 2025. – arXiv:2509.04715. – Текст : электронный. – URL: <https://arxiv.org/abs/2509.04715> (дата обращения: 01.12.2025).

13. Liu X., Chen M., Li C., Tian J., Zhou H., Ullah I. *An Improved UAV Detection Method Based on YOLOv5* // *Advanced Intelligent Computing Technology and Applications. ICIC 2023. Lecture Notes in Computer Science*. – Springer, 2023. – Т. 14086. – Текст : электронный. – URL: https://www.researchgate.net/publication/372754250_An_Improved_UAV_Detection_Method_Based_on_YOLOv5 (дата обращения: 01.12.2025).

14. Liu Z., Fan K., Chen Y., et. al. *Deep Learning-Based Acoustic Recognition of UAVs in Complex Environments* // *Drones*. – 2025. – Т. 9. – №. 6. – С. 389. – Текст : электронный. – URL: <https://www.mdpi.com/2504-446X/9/6/389> (дата обращения: 01.12.2025).

15. Redmon J., Divvala S., Girshick R., Farhadi A. *You Only Look Once: Unified, Real-Time Object Detection* // *arXiv preprint*. – 2016. – arXiv:1506.02640. – Текст : электронный. – URL: <https://arxiv.org/abs/1506.02640> (дата обращения: 01.12.2025).

16. Redmon J., Farhadi A. *YOLOv3: An Incremental Improvement* // *arXiv preprint*. – 2018. – arXiv:1804.02767. – Текст : электронный. – URL: <https://arxiv.org/abs/1804.02767> (дата обращения: 01.12.2025).

17. Ren J., Wang Z., Zhao J., Liu X., *A Drone Sound Recognition Approach Using Adaptive Feature Fusion and Cross-Attention Feature Enhancement* // *Electronics*. – 2025. – Т.

14. – №. 8. – С. 1491. – Текст : электронный. – URL: <https://www.mdpi.com/2079-9292/14/8/1491> (дата обращения: 01.12.2025).
18. Ren S., He K., Girshick R., Sun J. *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks* // *arXiv preprint*. – 2016. – arXiv:1506.01497. – Текст : электронный. – URL: <https://arxiv.org/abs/1506.01497> (дата обращения: 01.12.2025).
19. Seidaliyeva U., Ipbayeva L., Taissariyeva K., et al. *Advances and Challenges in Drone Detection and Classification Techniques: A State-of-the-Art Review* // *Sensors*. – 2024. – Vol. 24, No. 1. – Article 125. – Текст : электронный. – URL: <https://www.mdpi.com/1424-8220/24/1/125> (дата обращения: 01.12.2025).
20. Semenyuk V., Kurmashev I., Lupidi A., et al. *Advance and Refinement: The Evolution of UAV Detection and Classification Technologies* // *arXiv preprint*. – 2024. – arXiv:2409.05985. – Текст : электронный. – URL: <https://arxiv.org/abs/2409.05985> (дата обращения: 01.12.2025).
21. Sun Y., Liu Y., Wang L., et al. *Anti-UAV detection and identification technology: Fundamentals, methods and challenges* // *Physical Communication*. – 2025. – С. 102676. – Текст : электронный. – URL: <https://www.sciencedirect.com/science/article/pii/S0924424724006708> (дата обращения: 01.12.2025).
22. Tan M., Pang R., Le Q. V. *EfficientDet: Scalable and Efficient Object Detection* // *arXiv preprint*. – 2020. – arXiv:1911.09070. – Текст : электронный. – URL: <https://arxiv.org/abs/1911.09070> (дата обращения: 01.12.2025).
23. Tang G., Ni J., Zhao Y., et al. *A survey of object detection for UAVs based on deep learning* // *Remote Sensing*. – 2023. – Т. 16. – №. 1. – С. 149. – Текст : электронный. – URL: <https://www.mdpi.com/2072-4292/16/1/149> (дата обращения: 01.12.2025).
24. Xiao Z., Yang Y., Yuan S., et al. *AV-DTEC: Self-Supervised Audio-Visual Fusion for Drone Trajectory Estimation and Classification* // *arXiv preprint*. – 2024. – arXiv:2412.16928. – Текст : электронный. – URL: <https://arxiv.org/abs/2412.16928> (дата обращения: 01.12.2025).
25. Wu R., Wang H., Chen H.-T., Carneiro G. *Deep Multimodal Learning with Missing Modality: A Survey* // *arXiv preprint*. – 2024. – arXiv:2409.07825. – Текст : электронный. – URL: <https://arxiv.org/abs/2409.07825> (дата обращения: 01.12.2025).
26. *Ultralytics YOLOv8* : документация. – Текст : электронный. – URL: <https://docs.ultralytics.com> (дата обращения: 12.05.2026).
27. Zhang Y., Sun P., Jiang Y. et al. *ByteTrack: Multi-Object Tracking by Associating Every Detection Box* // *arXiv preprint*. – 2022. – arXiv:2110.06864. – Текст : электронный. – URL: <https://arxiv.org/abs/2110.06864> (дата обращения: 12.05.2026). – Репозиторий: <https://github.com/ifzhang/ByteTrack>.

28. *Apache Kafka* : документация. – Текст : электронный. – URL: <https://kafka.apache.org/documentation/> (дата обращения: 12.05.2026).
29. 29.McFee B., Raffel C., Liang D. et al. *librosa: Audio and Music Signal Analysis in Python* // *Proceedings of the 14th Python in Science Conference*. – 2015. – Текст : электронный. – URL: <https://librosa.org> (дата обращения: 12.05.2026).
30. Zhao J., Zhang J., Li D., Wang D. *Vision-based Anti-UAV Detection and Tracking (DUT Anti-UAV)* // *arXiv preprint*. – 2022. – arXiv:2205.10851. – Текст : электронный. – URL: <https://arxiv.org/abs/2205.10851> (дата обращения: 12.05.2026). – Репозиторий: <https://github.com/wangdongdut/DUT-Anti-UAV>.
31. *Drone-vs-Bird Detection Challenge (WOSDETC)* : датасет и материалы соревнования. – Текст : электронный. – URL: <https://github.com/wosdetc/challenge> (дата обращения: 12.05.2026).
32. Al-Emadi S., Al-Ali A., Al-Ali A. *Audio-Based Drone Detection and Identification Using Deep Learning Techniques (DroneAudioDataset)* // *2021 International Wireless Communications and Mobile Computing (IWCMC)*. – 2021. – Текст : электронный. – URL: <https://github.com/saraalemadi/DroneAudioDataset> (дата обращения: 12.05.2026).
33. Piczak K. J. *ESC: Dataset for Environmental Sound Classification* // *Proceedings of the 23rd ACM International Conference on Multimedia*. – 2015. – С. 1015–1018. – Текст : электронный. – URL: <https://github.com/karolpiczak/ESC-50> (дата обращения: 12.05.2026).
34. Gemmeke J. F., Ellis D. P. W., Freedman D. et al. *AudioSet: An Ontology and Human-Labeled Dataset for Audio Events* // *Proceedings of the IEEE ICASSP*. – 2017. – С. 776–780. – Текст : электронный. – URL: <https://research.google.com/audioset/> (дата обращения: 12.05.2026).
35. Yuan S., Yang Y., Nguyen T. H. et al. *MMAUD: A Comprehensive Multi-Modal Anti-UAV Dataset for Modern Miniature Drone Threats* // *arXiv preprint*. – 2024. – arXiv:2402.03706. – Текст : электронный. – URL: <https://arxiv.org/abs/2402.03706> (дата обращения: 12.05.2026). – Репозиторий: <https://github.com/ntu-aris/MMAUD>.
36. M. Aleksyuk, D. Tereshkin, A. Shaburov and M. Ushankova, "Detecting and Monitoring UAVs Using YOLOv8 and Tracking Algorithms," *2025 27th International Conference on Digital Signal Processing and its Applications (DSPA), Moscow, Russian Federation, 2025*, pp. 1-4, doi: 10.1109/DSPA64310.2025.10977858.
37. 37.Gong Y., Chung Y.-A., Glass J. *AST: Audio Spectrogram Transformer* // *Proceedings of Interspeech*. – 2021. – С. 571–575. – arXiv:2104.01778. – Текст : электронный. – URL: <https://arxiv.org/abs/2104.01778> (дата обращения: 12.05.2026). – (Используется как

backbone акустического детектора в режиме *arch = ast*; конкретные веса – *Rashidbm/samid-drone-detector* на *HuggingFace Hub*.)

38. Park D. S., Chan W., Zhang Y. et al. *SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition* // *Proceedings of Interspeech*. – 2019. – С. 2613–2617. – *arXiv*:1904.08779. – Текст : электронный. – URL: <https://arxiv.org/abs/1904.08779> (дата обращения: 12.05.2026).

Приложение 1

Графические модели и графики метрик

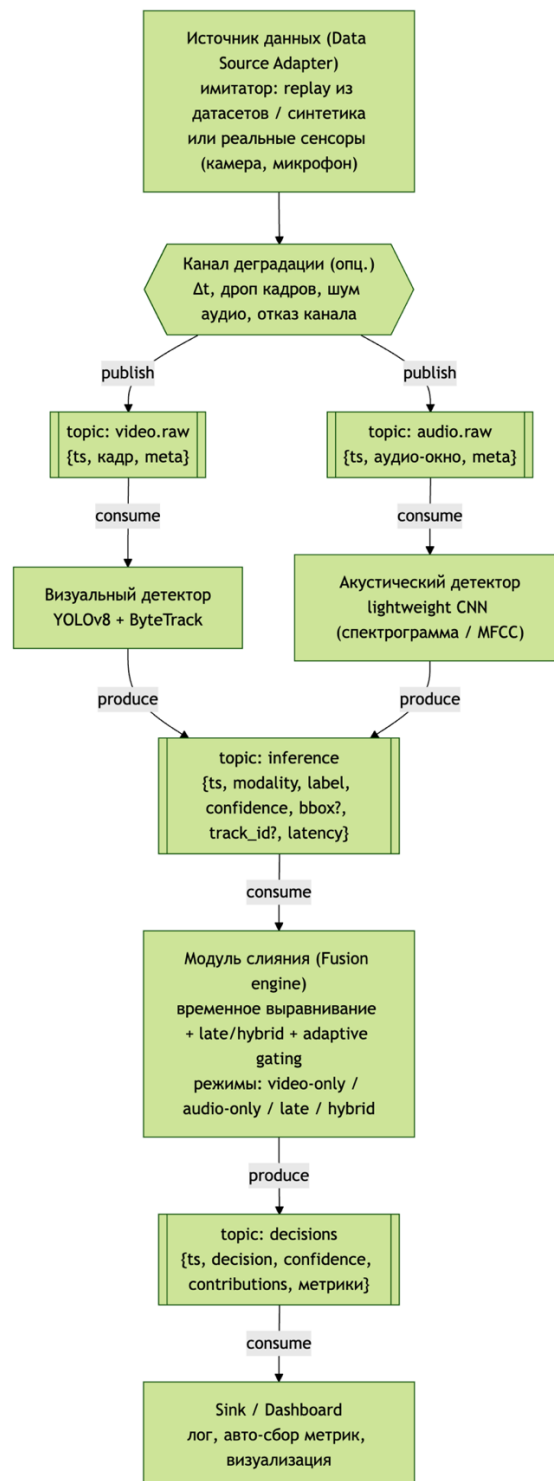


Рисунок 1 – Архитектура системы (поточковый пайплайн на брокере сообщений *Apache Kafka*)

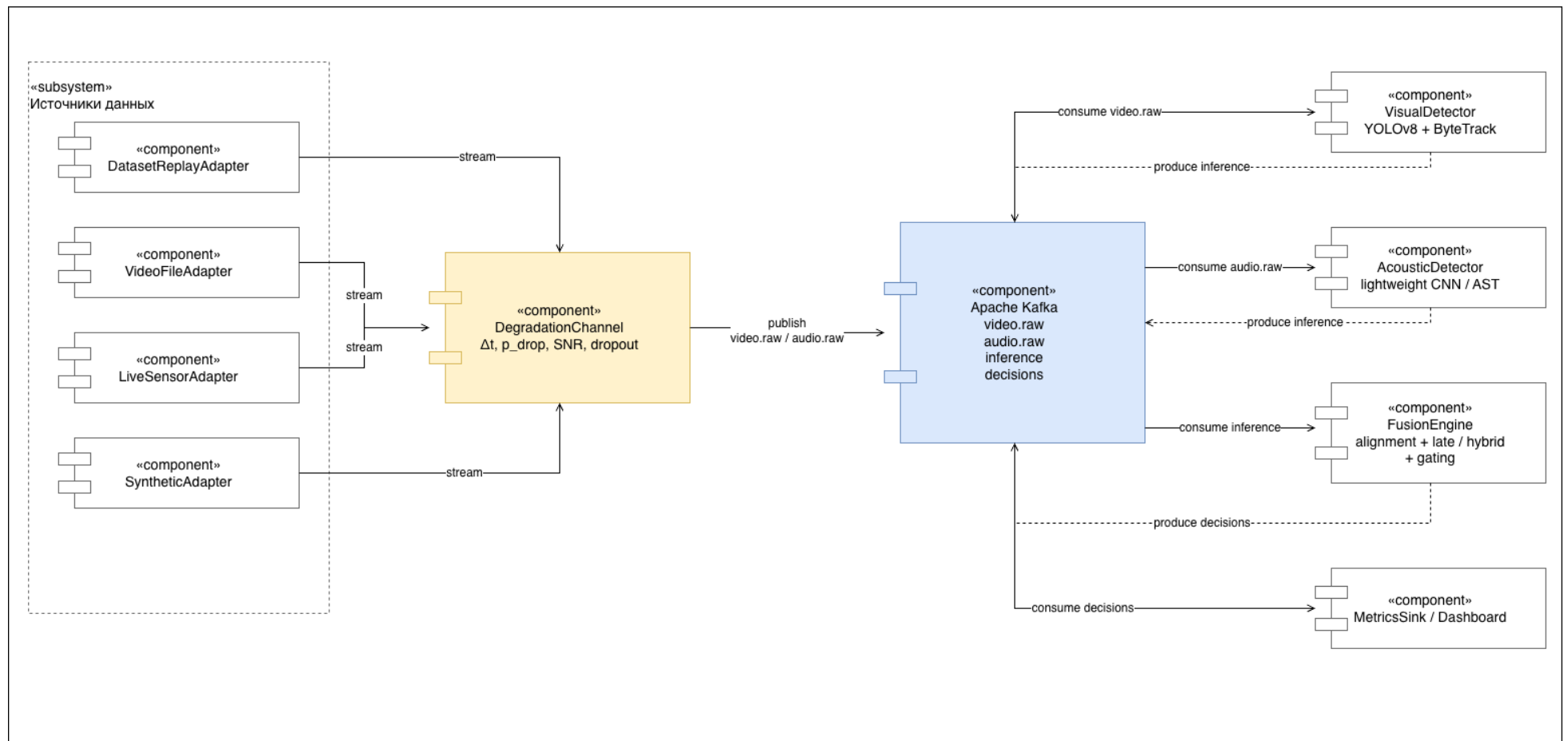


Рисунок 2 – *UML*-диаграмма компонентов системы

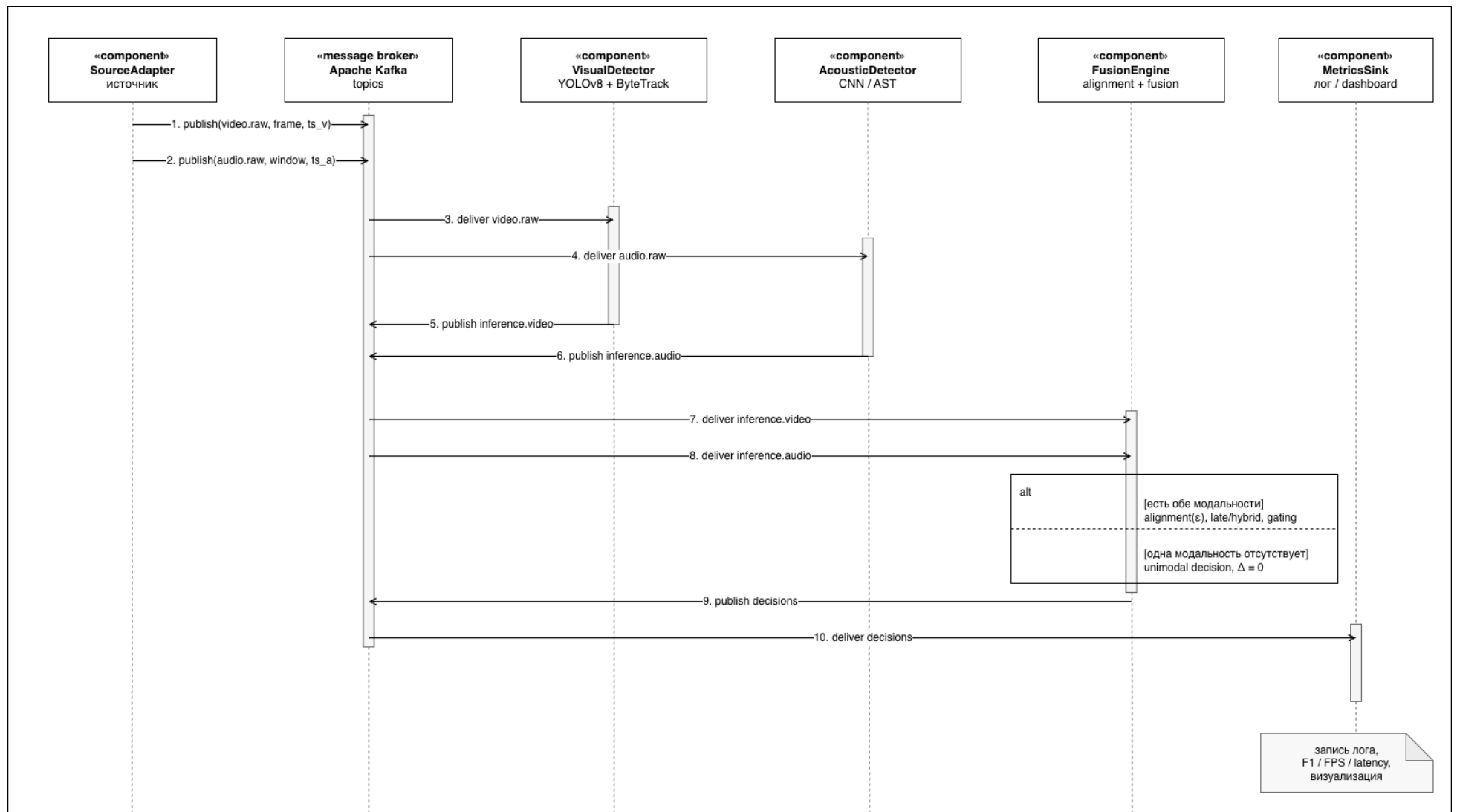


Рисунок 3 – UML-диаграмма последовательности обработки одного события

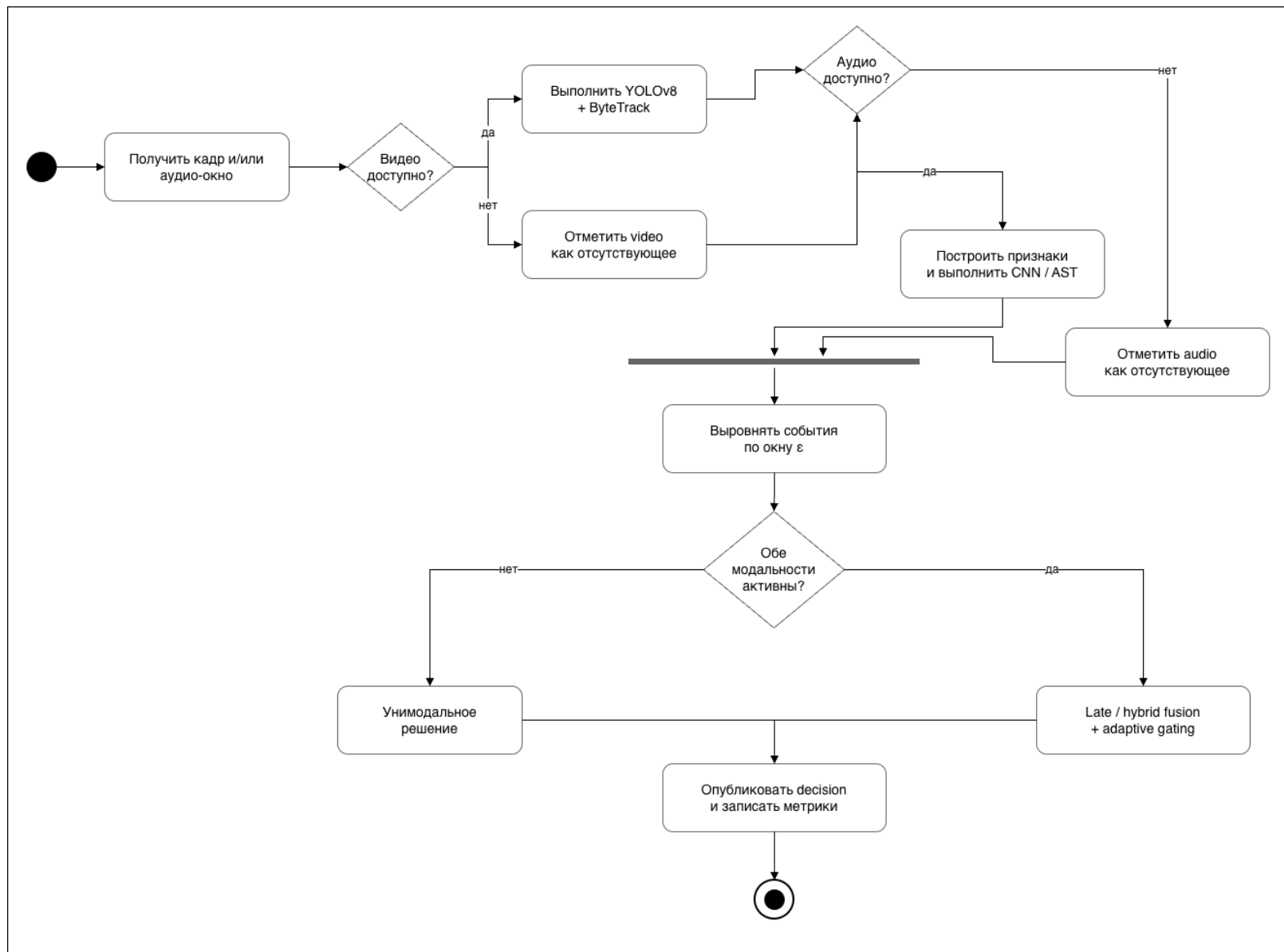


Рисунок 4 – *UML*-диаграмма деятельности обработки данных

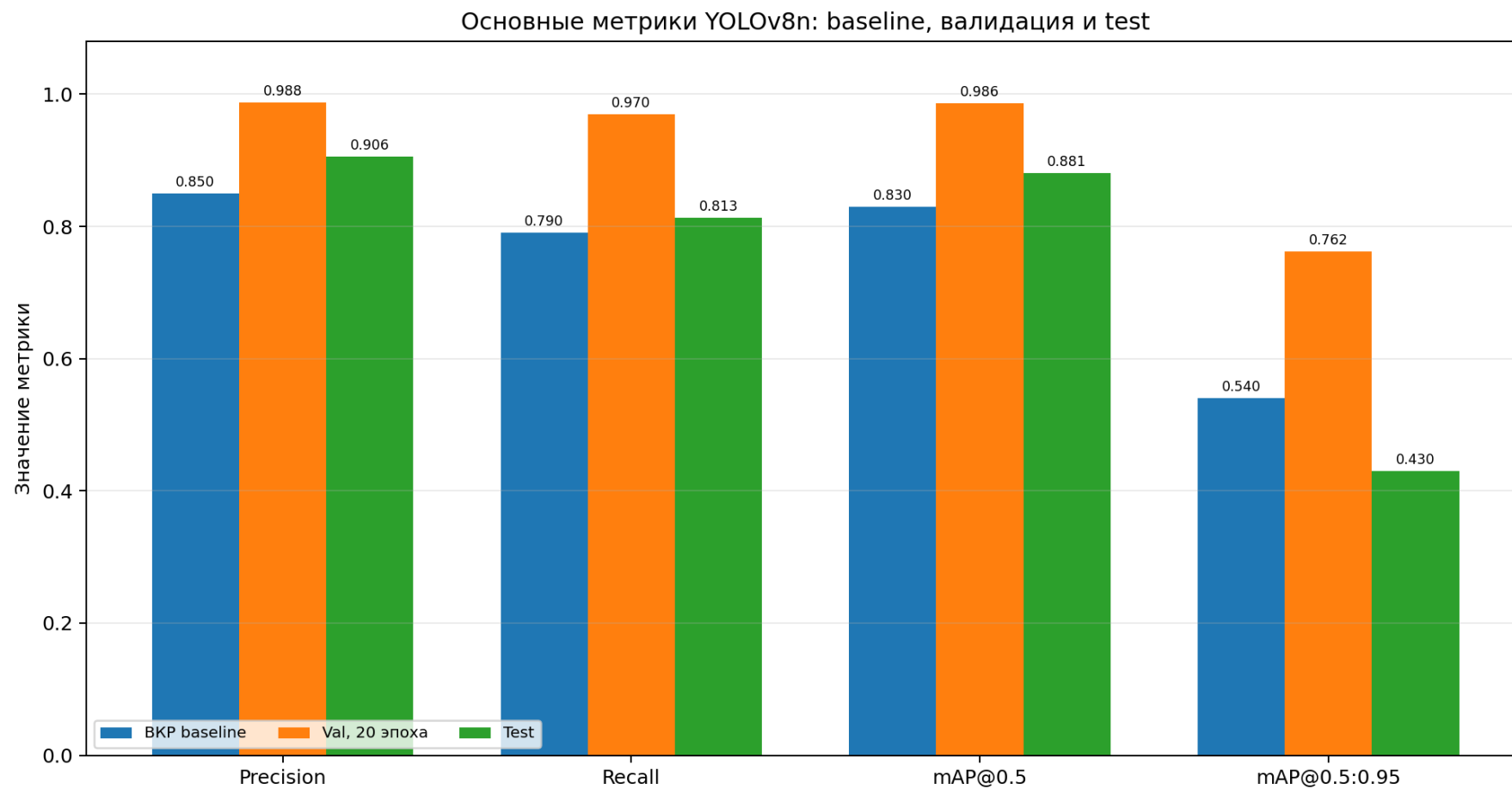


Рисунок 5 – График основных метрик дообучения *YOLOv8n* по контрольным точкам

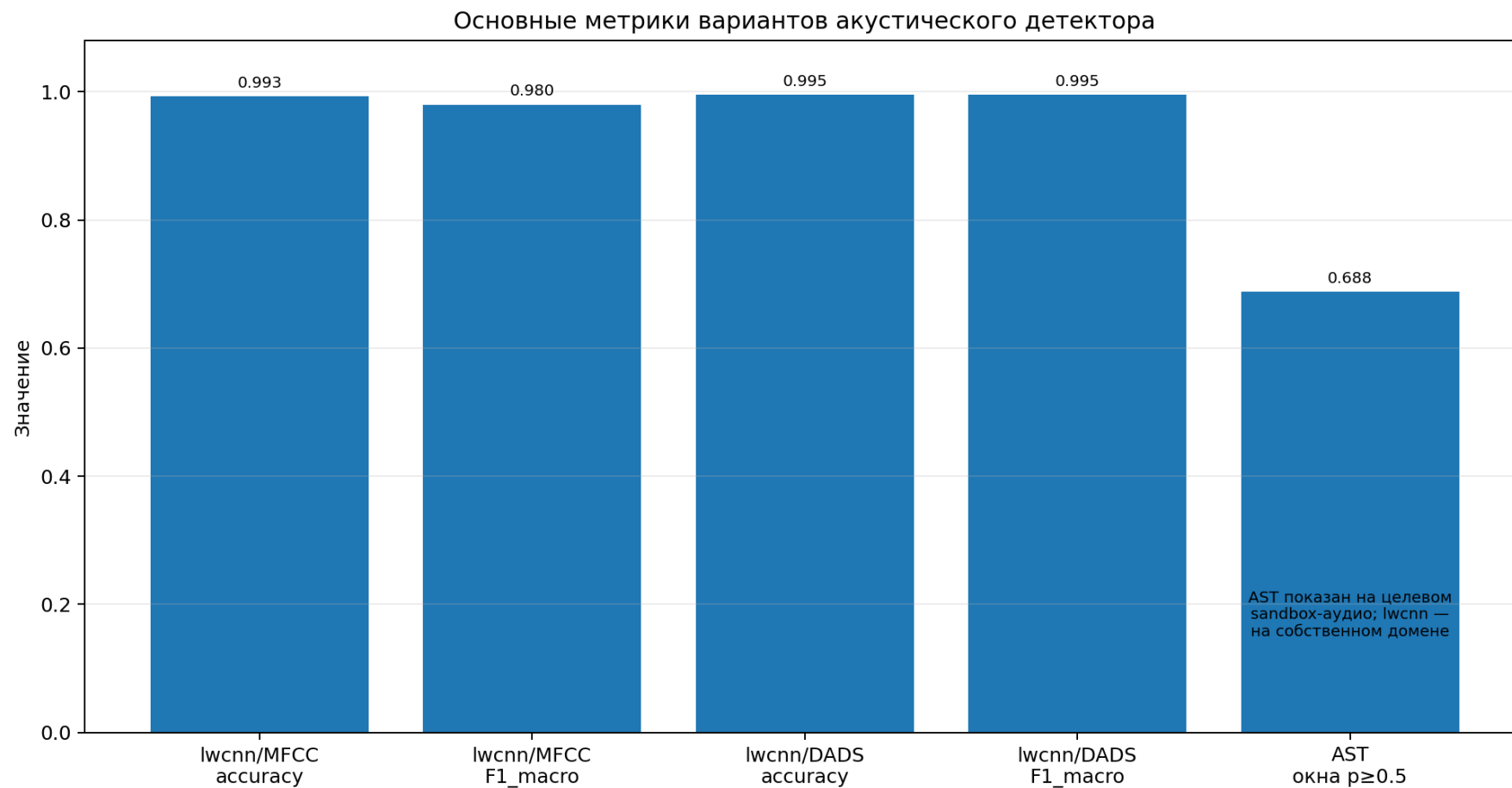


Рисунок 6 – График основных метрик вариантов акустического детектора

Приложение 2

Листинги ключевых компонентов системы

Приложение содержит листинги ключевых компонентов системы:

- адаптеры источников – *MediaFileAdapter* / *DatasetReplayAdapter* (Б.1, Б.2);
- детекторы-консьюмеры – визуальный (*YoloDetector* + *ByteTrackTracker*) и акустический (*LightweightCnnDetector* / *AstAudioClassifier*) (Б.3–Б.6);
- модуль слияния – стратегии *LateFusion* / *HybridFusion*, политика *AdaptiveGating* (Б.7–Б.9);
- «канал деградации» – стратегии *DtShift* / *FrameDrop* / *WindowDrop* / *AudioNoise* / *ModalityDropout* / *ConfidenceJitter* (Б.10).

Листинги приведены на языке *Python* 3.11+ (с типизированными аннотациями, *dataclasses*). Полные исходные файлы – в репозитории проекта (github.com/Yagiar/MasterDiploma); ниже – ключевые методы и контракты, опущены импорты и второстепенные служебные блоки.

2.1 Адаптер источника *MediaFileAdapter* (видео и аудио)

Воспроизводит видеофайл покадрово и аудио-окнами фиксированной длины с заданным шагом (*sliding window*) из отдельного *.wav*. Используется для «тепличного» сценария с синхронными модальностями (раздел 6.1).

class *MediaFileAdapter*:

"""Адаптер «видеофайл (+ опц. аудио из wav)»."""

name = "*media_file*"

```
def __init__(self, *, source_id, video_path, audio_path=None,
             fps=25.0, loop=True, jpeg_quality=75,
             audio_sample_rate=16000, audio_win_ms=1000, audio_hop_ms=500):
    self._source_id = source_id
    self._video_path = video_path
    self._audio_path = audio_path # None -> только видео
    self._loop = loop
    self._jpeg_quality = jpeg_quality
    self._requested_fps = float(fps)
    self._sr, self._win_ms, self._hop_ms = audio_sample_rate, audio_win_ms, audio_hop_ms
    self._cap = None
```

@property

def target_period_s(self) -> float:

"""Период между видеокадрами (=1/fps)."""

return 1.0 / self._requested_fps **if** self._requested_fps > 0 **else** 0.0

@property

def audio_period_s(self) -> float:

```

"""Период между аудио-окнами (= hop)."""
return self._hop_ms / 1000.0

def frames(self):
    """Итератор кадров: декодируем видео, кодируем в JPEG, выдаём FrameItem."""
    self._cap = cv2.VideoCapture(self._video_path)
    seq = 0
    params = [int(cv2.IMWRITE_JPEG_QUALITY), self._jpeg_quality]
    while True:
        ok, frame = self._cap.read()
        if not ok:
            if self._loop:
                self._cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
                continue
            break
        ok_enc, buf = cv2.imencode(".jpg", frame, params)
        if not ok_enc:
            continue
        h, w = frame.shape[:2]
        seq += 1
        yield FrameItem(jpeg_bytes=buf.tobytes(), seq=seq, width=w, height=h,
                        fps_nominal=self._fps_nominal, meta={"source_kind": "media_file"})

def audio_windows(self):
    """Итератор аудио-окон: грузим wav как float32 моно на целевой sample_rate,
    режем sliding window (win_ms / hop_ms), кодируем в PCM int16."""
    if self._audio_path is None:
        return
    signal = self._load_audio_mono()
    win = int(self._sr * self._win_ms / 1000)
    hop = int(self._sr * self._hop_ms / 1000)
    seq = 0
    while True:
        offset = 0
        while offset + win <= signal.size:
            seq += 1
            yield AudioItem(
                pcm_bytes=self._float_to_pcm16(signal[offset : offset + win]),
                seq=seq, sample_rate=self._sr, channels=1,
                len_ms=self._win_ms, hop_ms=self._hop_ms,
                meta={"source_kind": "media_file"},
            )
            offset += hop
        if not self._loop:
            return

    @staticmethod
    def _float_to_pcm16(x: np.ndarray) -> bytes:
        return (np.clip(x, -1.0, 1.0) * 32767.0).astype("<i2").tobytes()

```

2.2 Адаптер источника *DatasetReplayAdapter*

Базовый адаптер «replay одного видеофайла» – только видеомодалность. Используется для прогонов на публичных видеодатасетах (*DUT Anti-UAV* [30], *Drone-vs-Bird* [31]) и как ссылочный пример простейшего источника.

```
class DatasetReplayAdapter(NullAudioMixin):
    """Адаптер replay одного видеофайла (без аудио)."""

    name = "dataset_replay"

    def __init__(self, *, source_id, video_path, fps=25.0, loop=True, jpeg_quality=75):
        self._source_id = source_id
        self._video_path = video_path
        self._loop = loop
        self._jpeg_quality = jpeg_quality
        self._requested_fps = float(fps)
        self._cap = None

    def frames(self):
        self._cap = cv2.VideoCapture(self._video_path)
        seq = 0
        encode_params = [int(cv2.IMWRITE_JPEG_QUALITY), self._jpeg_quality]
        while True:
            ok, frame = self._cap.read()
            if not ok:
                if self._loop:
                    self._cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
                    continue
                break
            ok_enc, buf = cv2.imencode(".jpg", frame, encode_params)
            if not ok_enc:
                continue
            height, width = frame.shape[:2]
            seq += 1
            yield FrameItem(jpeg_bytes=buf.tobytes(), seq=seq, width=width, height=height,
                           fps_nominal=self._requested_fps, meta={"source_kind": "dataset_replay"})
```

2.3 Визуальный детектор *YoloDetector* (YOLOv8)

Обёртка над *Ultralytics YOLOv8*: грузит веса (*yolov8s-uav.pt* – раздел 6.2), на кадре возвращает список детекций с уверенностями и боксами в формате $[x, y, w, h]$. Метка «дрон» определяется либо явным списком *drone_class_ids* (из конфигурации), либо по имени класса.

```
@dataclass(frozen=True)
class Detection:
    label: str # "drone" | "non-drone"
    confidence: float
    bbox: list[float] # [x, y, w, h] в пикселях исходного кадра

class YoloDetector:
    """Детектор на Ultralytics YOLOv8."""
```

```

def __init__(self, *, weights_path, conf_threshold=0.25, iou_threshold=0.45,
              device="cpu", imgsz=640, drone_class_ids=None):
    from ultralytics import YOLO
    self.model = YOLO(weights_path)
    self._conf, self._iou, self._device, self._imgsz = conf_threshold, iou_threshold, device, imgsz
    self._names = dict(getattr(self.model, "names", {})) or {}
    self._drone_ids = set(drone_class_ids) if drone_class_ids else None

def _label_for(self, cls_id: int) -> str | None:
    if self._drone_ids is not None:
        return "drone" if cls_id in self._drone_ids else "non-drone"
    name = self._names.get(cls_id, "").lower()
    return "drone" if ("drone" in name or "uav" in name) else "non-drone"

def detect(self, frame: np.ndarray) -> DetectResult:
    t0 = time.perf_counter()
    results = self._model.predict(frame, conf=self._conf, iou=self._iou,
                                  device=self._device, imgsz=self._imgsz, verbose=False)
    latency_ms = (time.perf_counter() - t0) * 1000.0
    detections = []
    for res in results:
        xyxy = res.bboxes.xyxy.cpu().numpy()
        confs = res.bboxes.conf.cpu().numpy()
        clss = res.bboxes.cls.cpu().numpy()
        for (x1, y1, x2, y2), conf, cls_id in zip(xyxy, confs, clss):
            label = self._label_for(int(cls_id))
            if label is None:
                continue
            detections.append(Detection(label=label, confidence=float(conf),
                                       bbox=[float(x1), float(y1),
                                             float(x2 - x1), float(y2 - y1)]))
    return DetectResult(detections=detections, latency_ms=latency_ms,
                       model_name=self._model_name, model_ver=self._model_ver)

```

2.4 Алгоритм трекинга *ByteTrackTracker*

Обёртка над *supervision.ByteTrack* с состоянием на каждый *source_id*. После каждой детекции кадра сопоставляет вход с возвращёнными *tracked*-боксами по *IoU* (порог 0.3), а не по индексу: *ByteTrack* может отбрасывать/переупорядочивать детекции, поэтому индексное сопоставление давало бы неверные *track_id* (дефект, найденный и исправленный в прогоне №2 – раздел 6.3).

```

class ByteTrackTracker:
    """Обёртка над supervision.ByteTrack с состоянием per source_id."""

    def __init__(self):
        import supervision as sv
        self._sv = sv
        self._trackers: dict[str, "sv.ByteTrack"] = {}

```

```

def _tracker_for(self, source_id):
    tr = self._trackers.get(source_id)
    if tr is None:
        tr = self._sv.ByteTrack()
        self._trackers[source_id] = tr
    return tr

@staticmethod
def iou(a: np.ndarray, b: np.ndarray) -> float:
    x1, y1 = max(a[0], b[0]), max(a[1], b[1])
    x2, y2 = min(a[2], b[2]), min(a[3], b[3])
    inter = max(0.0, x2 - x1) * max(0.0, y2 - y1)
    area_a = max(0.0, a[2] - a[0]) * max(0.0, a[3] - a[1])
    area_b = max(0.0, b[2] - b[0]) * max(0.0, b[3] - b[1])
    union = area_a + area_b - inter
    return float(inter / union) if union > 0 else 0.0

def update(self, source_id, detections):
    """Прогнать детекции кадра через трекер, вернуть пары (детекция, track_id|None)."""
    if not detections:
        return []
    sv = self._sv
    in_xyxy = np.array(
        [[d.bbox[0], d.bbox[1], d.bbox[0] + d.bbox[2], d.bbox[1] + d.bbox[3]]
         for d in detections], dtype=float)
    det = sv.Detections(xyxy=in_xyxy,
                        confidence=np.array([d.confidence for d in detections]),
                        class_id=np.zeros(len(detections), dtype=int))
    tracked = self._tracker_for(source_id).update_with_detections(det)
    tr_xyxy = np.asarray(getattr(tracked, "xyxy", np.empty((0, 4))))
    tr_ids = getattr(tracked, "tracker_id", None)
    tr_ids = np.asarray(tr_ids) if tr_ids is not None else np.empty((0,), dtype=int)

    out = []
    for i, d in enumerate(detections):
        tid = None
        if tr_xyxy.shape[0] > 0:
            ious = np.array([self.iou(in_xyxy[i], tr_xyxy[j])
                             for j in range(tr_xyxy.shape[0])])
            j = int(np.argmax(ious))
            if ious[j] >= 0.3 and j < tr_ids.shape[0]:
                tid = int(tr_ids[j])
            out.append((d, tid))
    return out

```

2.5 Акустический детектор *LightweightCnnDetector (lwcnn)*

Облегчённая *CNN*-классификация спектрограмм «дрон / не-дрон» – три свёрточных блока (*Conv*–*BN*–*ReLU*–*MaxPool*) + *GAP* + *Linear*. Архитектура *LightweightAudioCNN* синхронизирована с обучающим пайплайном *train/uavtrain/audio_models.py*. Если файл

весов отсутствует – обёртка переходит в режим энергетического порога (для запуска без обучения).

```
DRONE_CLASS_INDEX = 1 # AUDIO_CLASSES = ["non-drone", "drone"]
```

```
def build_lwcnn(n_classes: int):
    """LightweightAudioCNN – 3 свёрточных блока + GAP + Linear."""
    from torch import nn

    class LightweightAudioCNN(nn.Module):
        def __init__(self, n_cls):
            super().__init__()
            self.features = nn.Sequential(
                nn.Conv2d(1, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(inplace=True), nn.MaxPool2d(2),
                nn.Conv2d(16, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(inplace=True), nn.MaxPool2d(2),
                nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(inplace=True), nn.AdaptiveAvgPool2d(1),
            )
            self.classifier = nn.Linear(64, n_cls)

        def forward(self, x):
            return self.classifier(self.features(x).flatten(1))

    return LightweightAudioCNN(n_classes)
```

```
class LightweightCnnDetector:
    """Акустический детектор: CNN по спектрограмме либо энергетический порог."""
```

```
    def __init__(self, *, weights_path, feature_dim, n_frames,
                 device="cpu", energy_threshold=0.01, arch="lwcnn"):
        self._weights_path = Path(weights_path)
        self._device = device
        self._energy_threshold = energy_threshold
        self._arch = arch
        self._backend = "energy-threshold"
        self._model = None
        if self._weights_path.exists():
            self._load_cnn(feature_dim, n_frames)

    def _load_cnn(self, feature_dim, n_frames):
        import torch
        model = build_audio_model(self._arch, feature_dim=feature_dim,
                                  n_frames=n_frames, n_classes=2)
        model.load_state_dict(torch.load(str(self._weights_path), map_location=self._device))
        model.eval().to(self._device)
        self._model = model
        self._backend = "cnn"

    def detect(self, features_array: np.ndarray) -> AudioDetection:
        if self._backend == "cnn":
```

```

import torch
with torch.no_grad():
    x = torch.from_numpy(features_array).float().unsqueeze(0).unsqueeze(0).to(self._device)
e)
    probs = torch.softmax(self._model(x), dim=1).cpu().numpy().ravel()
    p_drone = float(probs[DRONE_CLASS_INDEX])
    return AudioDetection(label="drone" if p_drone >= 0.5 else "non-drone",
                          confidence=p_drone if p_drone >= 0.5 else 1.0 - p_drone,
                          backend="cnn")
# энергетический порог (заглушка без весов)
energy = float(np.mean(features_array.astype(np.float64) ** 2))
is_drone = energy >= self._energy_threshold
conf = 0.5 + 0.5 * min(1.0, abs(energy - self._energy_threshold) /
                       max(self._energy_threshold, 1e-6))
return AudioDetection(label="drone" if is_drone else "non-drone",
                      confidence=conf, backend="energy-threshold")

```

2.6 Акустический детектор *AstAudioClassifier* (AST)

Обёртка над предобученным *Audio Spectrogram Transformer* [37] (*HuggingFace Rashidbm/samid-drone-detector*). Принимает *raw pcm* (минуя *FeatureExtractor lwcnn*-режима) и использует собственный *ASTFeatureExtractor*. Маршрутизация – флаг *acoustic_detector.arch=ast* в конфигурации сервиса. Закрывает *domain gap* на «полевом» аудио (раздел 6.4).

class *AstAudioClassifier*:

```

"""Audio Spectrogram Transformer для бинарной задачи 'дрон / не-дрон'."""

expects_pcm = True # маркер для AudioConsumer: передавать сырой PCM, не наши features

def __init__(self, *, weights_path, device="cpu", target_sample_rate=16000):
    self._weights_path = weights_path
    self._device = device
    self._target_sr = target_sample_rate
    self._model = None
    self._fe = None
    self._id2label = {0: "no_drone", 1: "drone"}
    self._backend = "energy-threshold"
    try:
        self._load(weights_path)
    except Exception:
        pass # fallback на energy-threshold, если модели нет

def _load(self, src: str) -> None:
    import torch
    from transformers import AutoFeatureExtractor, AutoModelForAudioClassification
    self._fe = AutoFeatureExtractor.from_pretrained(src)
    model = AutoModelForAudioClassification.from_pretrained(src)
    model.eval().to(self._device)
    self._model = model

```

```

self._id2label = {int(k): v for k, v in (model.config.id2label or {}).items()}
self._backend = "ast"

def detect(self, payload_b64, *, src_sample_rate, channels) -> AudioDetection:
    if self._backend != "ast":
        return AudioDetection(label="non-drone", confidence=0.5, backend="energy-threshold")
    )

    import torch
    # PCM int16 LE -> float32 [-1, 1], ресемпл к target_sr
    sig = _pcm_int16_to_float(b64decode_str(payload_b64), channels)
    if src_sample_rate != self._target_sr and sig.size > 0:
        import librosa
        sig = librosa.resample(sig, orig_sr=src_sample_rate, target_sr=self._target_sr)
    # ASTFeatureExtractor: PCM -> мел-спектрограмма (128 мел, max_length=1024)
    inputs = self._fe(sig, sampling_rate=self._target_sr, return_tensors="pt")
    inputs = {k: v.to(self._device) for k, v in inputs.items()}
    with torch.no_grad():
        probs = torch.softmax(self._model(**inputs).logits, dim=-1).cpu().numpy().ravel()
        drone_idx = next((i for i, lbl in self._id2label.items()
                           if str(lbl).lower() in ("drone", "yes_drone", "uav")), 1)
        p_drone = float(probs[drone_idx])
        return AudioDetection(label="drone" if p_drone >= 0.5 else "non-drone",
                              confidence=p_drone if p_drone >= 0.5 else 1.0 - p_drone,
                              backend="ast")

```

2.7 Стратегия слияния *LateFusion*

Реализует модель позднего слияния (раздел 4.2.1): взвешенная сумма уверенностей по активным модальностям + правило компенсации Δ . Веса перенормируются по активным каналам (отсутствующая модальность не «топит» решение). Правило компенсации: $+\delta_{conf}$ при взаимном подтверждении, $-\delta_{unconf}$ при противоречии каналов, иначе $\Delta = 0$.

```

def _effective_weights(w_v, w_a, v_active, a_active):
    """Перенормировать веса по активным каналам (сумма по активным = 1)."""
    wv = w_v if v_active else 0.0
    wa = w_a if a_active else 0.0
    s = wv + wa
    if s <= 0:
        n = (1 if v_active else 0) + (1 if a_active else 0)
        return (1.0 / n if v_active and n else 0.0,
                1.0 / n if a_active and n else 0.0)
    return wv / s, wa / s

```

`@dataclass`

`class LateFusion:`

"""Позднее слияние с перенормировкой весов и правилом компенсации Δ ."""

```

delta_conf: float = 0.1 # бонус за взаимное подтверждение
delta_unconf: float = 0.1 # штраф за противоречие
label_threshold: float = 0.5 # порог 'drone' для проверки согласия каналов
mode: str = "late"

```

```

def _says_drone(self, msg) -> bool:
    return (msg is not None and msg.label == "drone"
            and msg.confidence >= self.label_threshold)

def fuse(self, window: AlignedWindow, *, w_v, w_a, threshold) -> FusionOutcome | None:
    best_v = window.best_video()
    best_a = window.best_audio()
    if best_v is None and best_a is None:
        return None

    v_active, a_active = best_v is not None, best_a is not None
    p_v = (best_v.confidence if best_v and best_v.label == "drone" else 0.0) if best_v else 0.0
    p_a = (best_a.confidence if best_a and best_a.label == "drone" else 0.0) if best_a else 0.0
    v_drone, a_drone = self._says_drone(best_v), self._says_drone(best_a)

    delta = 0.0
    if v_active and a_active:
        if v_drone and a_drone:
            delta = self.delta_conf # +δ_conf – взаимное подтверждение
        elif v_drone ^ a_drone:
            delta = -self.delta_unconf # -δ_unconf – противоречие каналов

    eff_w_v, eff_w_a = _effective_weights(w_v, w_a, v_active, a_active)
    p_fused = _clip01(eff_w_v * p_v + eff_w_a * p_a + delta)

    return FusionOutcome(p_fused=p_fused, decision=p_fused >= threshold,
                        p_v=p_v if v_active else None,
                        p_a=p_a if a_active else None,
                        w_v=eff_w_v, w_a=eff_w_a, delta=delta,
                        source_msg_ids=tuple(m.msg_id for m in (best_v, best_a) if m))

```

2.8 Стратегия слияния *HybridFusion*

Прагматичный гибрид на уровне решений с нелинейной комбинацией и адаптивными весами (раздел 2.2.3). При согласии обоих активных каналов берётся максимум из взвешенной суммы и «вероятностного ИЛИ» $1 - (1 - p_v)(1 - p_a) + \delta_{conf}$; при противоречии – штраф $-\delta_{unconf}$; при одном активном – решение по нему без штрафа.

```

@dataclass
class HybridFusion:
    """Гибридное слияние (decision-level, нелинейное, с адаптивными весами)."""

```

```

    delta_conf: float = 0.1
    delta_unconf: float = 0.15
    label_threshold: float = 0.5
    mode: str = "hybrid"

    def _says_drone(self, msg) -> bool:
        return (msg is not None and msg.label == "drone"
                and msg.confidence >= self.label_threshold)

```

```

def fuse(self, window: AlignedWindow, *, w_v, w_a, threshold) -> FusionOutcome | None:
    best_v = window.best_video()
    best_a = window.best_audio()
    if best_v is None and best_a is None:
        return None

    v_active, a_active = best_v is not None, best_a is not None
    p_v = (best_v.confidence if best_v and best_v.label == "drone" else 0.0) if best_v else 0.0
    p_a = (best_a.confidence if best_a and best_a.label == "drone" else 0.0) if best_a else 0.0

    if v_active and not a_active:
        p_fused, delta, eff_w_v, eff_w_a = p_v, 0.0, 1.0, 0.0
    elif a_active and not v_active:
        p_fused, delta, eff_w_v, eff_w_a = p_a, 0.0, 0.0, 1.0
    else:
        eff_w_v, eff_w_a = w_v, w_a
        s = eff_w_v * p_v + eff_w_a * p_a
        v_drone, a_drone = self._says_drone(best_v), self._says_drone(best_a)
        if v_drone and a_drone:
            prob_or = 1.0 - (1.0 - p_v) * (1.0 - p_a)
            delta = self.delta_conf
            p_fused = max(s, prob_or) + delta # нелинейный буст
        elif v_drone ^ a_drone:
            delta = -self.delta_unconf
            p_fused = s + delta
        else:
            delta = 0.0
            p_fused = s

    p_fused = _clip01(p_fused)
    return FusionOutcome(p_fused=p_fused, decision=p_fused >= threshold,
        p_v=p_v if v_active else None,
        p_a=p_a if a_active else None,
        w_v=eff_w_v, w_a=eff_w_a, delta=delta,
        source_msg_ids=tuple(m.msg_id for m in (best_v, best_a) if m))

```

2.9 Политика взвешивания *AdaptiveGating*

Динамические веса w_v, w_a как функция качества каналов в окне (раздел 2.2.3): резкость / яркость кадра (подсказки `quality.img_sharpness` / `quality.img_brightness` от `visual-detector`), SNR аудио (`quality.snr_db` от `acoustic-detector`). Снижает вклад деградированного канала, не обнуляя его полностью (нижняя граница q_{floor}).

```

@dataclass(frozen=True)
class GatingResult:
    w_v: float
    w_a: float
    snr_audio: float | None = None
    img_quality: float | None = None

def _normalize(w_v, w_a):
    s = w_v + w_a

```

```

if s <= 0:
    return 0.5, 0.5
return w_v / s, w_a / s

@dataclass(frozen=True)
class AdaptiveGating:
    """Адаптивные веса по качеству каналов ( $q_{\text{floor}} \leq q \leq 1$ )."""

    base_w_v: float = 0.5
    base_w_a: float = 0.5
    q_floor: float = 0.2
    sharpness_ref: float = 150.0 # эталонная резкость (дисперсия лапласиана)
    snr_ref_db: float = 20.0 # эталонный SNR аудио
    snr_floor_db: float = 0.0

    def _img_quality(self, window: AlignedWindow) -> float | None:
        best_v = window.best_video()
        if best_v is None:
            return None
        sharp = best_v.quality.img_sharpness
        bright = best_v.quality.img_brightness
        q_sharp = 1.0 if sharp is None else max(0.0, min(1.0, sharp / self.sharpness_ref))
        q_bright = 1.0 if bright is None else max(0.0, 1.0 - 2.0 * abs(bright - 0.5))
        q = q_sharp * q_bright if (sharp is not None and bright is not None) else (q_sharp if sharp
is not None else q_bright)
        return max(self.q_floor, q)

    def _audio_quality(self, window: AlignedWindow):
        best_a = window.best_audio()
        if best_a is None or best_a.quality.snr_db is None:
            return None, None
        snr = best_a.quality.snr_db
        span = max(1e-6, self.snr_ref_db - self.snr_floor_db)
        q = (snr - self.snr_floor_db) / span
        return max(self.q_floor, min(1.0, q)), snr

    def weights(self, window: AlignedWindow) -> GatingResult:
        q_img = self._img_quality(window)
        q_aud, snr = self._audio_quality(window)
        eff_v = self.base_w_v * (q_img if q_img is not None else 1.0)
        eff_a = self.base_w_a * (q_aud if q_aud is not None else 1.0)
        wv, wa = _normalize(eff_v, eff_a)
        return GatingResult(w_v=wv, w_a=wa, snr_audio=snr, img_quality=q_img)

```

2.10 «Канал деградации» – стратегии

Реализация модели «канала деградации» (раздел 4.2.3): набор сменных стратегий (паттерн *Strategy*), композируемых *DegradationChannel*’ом (паттерн *Decorator*) поверх адаптера источника. На пилоте канал выключен (*source.degradation.enabled: false*); включается для отдельных стресс-прогонов (раздел 5.2). Все стратегии детерминированы при заданном *seed*.

$StreamItem = FrameItem \mid AudioItem$

@runtime_checkable

class DegradationStrategy(Protocol):

"""Контракт стратегии деградации."""

name: str

def apply(*self*, item: StreamItem) -> StreamItem | None: ... *# None = событие отброшено*

Δt_{shift} – сдвиг временной метки одной модальности на фиксированную величину Δt_{shift} (миллисекунды). Имитация рассинхронизации сенсоров; задаёт ось эксперимента по «временному выравниванию» (пробел 1 из 1.2).

@dataclass

class DtShift:

"""Сдвиг t_s указанной модальности на $shift_ms$ (рассинхрон Δt)."""

shift_ms: float = 0.0

target: str = "audio" # "video" | "audio"

name: str = "dt_shift"

def apply(*self*, item: StreamItem) -> StreamItem | None:

modality = "video" if isinstance(item, FrameItem) else "audio"

if modality != self.target:

return item

new_ts = (item.ts + self.shift_ms / 1000.0) if item.ts is not None else None

*new_meta = {**item.meta, "dt_shift_ms": str(self.shift_ms)}*

return _replace_ts(item, new_ts, new_meta)

$FrameDrop$ / $WindowDrop$ – случайный дроп видеокадров / аудио-окон с вероятностью p_{drop} . Имитация потерь в канале.

@dataclass

class FrameDrop:

"""Случайный дроп видеокадров (аудио не трогает)."""

drop_prob: float = 0.0

seed: int | None = None

name: str = "frame_drop"

def __post_init__(*self*):

self._rng = random.Random(self.seed)

def apply(*self*, item: StreamItem) -> StreamItem | None:

if isinstance(item, FrameItem) and self._rng.random() < self.drop_prob:

return None

return item

@dataclass

class WindowDrop:

"""Случайный дроп аудио-окон (видео не трогает)."""

drop_prob: float = 0.0

```
seed: int | None = None
name: str = "window_drop"
```

```
def __post_init__(self):
    self._rng = random.Random(self.seed)
```

```
def apply(self, item: StreamItem) -> StreamItem | None:
    if isinstance(item, AudioItem) and self._rng.random() < self.drop_prob:
        return None
    return item
```

AudioNoise – добавление гауссова шума в *pcm* аудио-окно до целевого SNR (дБ).

Уровень шума подбирается из энергии сигнала окна: $\sigma_{noise} = \text{RMS}_{signal} / 10^{SNR/20}$.

Имитация шумной обстановки.

@dataclass

class AudioNoise:

"""Добавление гауссова шума до целевого SNR (дБ); видео не трогает."""

```
target_snr_db: float = 20.0
seed: int | None = None
name: str = "audio_noise"
```

```
def __post_init__(self):
    self._rng = np.random.default_rng(self.seed)
```

```
def apply(self, item: StreamItem) -> StreamItem | None:
    if not isinstance(item, AudioItem):
        return item
    x = np.frombuffer(item.pcm_bytes, dtype="<i2").astype(np.float32) / 32768.0
    rms = float(np.sqrt(np.mean(x ** 2))) if x.size else 0.0
    noise_std = (rms / (10.0 ** (self.target_snr_db / 20.0))) if rms > 0 else 1e-3
    noisy = x + self._rng.normal(0.0, noise_std, size=x.shape).astype(np.float32)
    pcm = (np.clip(noisy, -1.0, 1.0) * 32767.0).astype("<i2").tobytes()
    return AudioItem(pcm_bytes=pcm, seq=item.seq, sample_rate=item.sample_rate,
                    channels=item.channels, len_ms=item.len_ms, hop_ms=item.hop_ms,
                    ts=item.ts,
                    meta={**item.meta, "snr_target": f"{self.target_snr_db:.1f}"})
```

ModalityDropout – полное отключение указанной модальности на интервалах $[t_a, t_b)$.

Имитация отказа сенсора; проверяет устойчивость к *missing modality* (пробел 4 из 1.2).

@dataclass

class ModalityDropout:

"""Полное отключение модальности на заданных интервалах [start_s, end_s)."""

```
modality: str = "audio" # "video" | "audio"
intervals_s: list[tuple[float, float]] = field(default_factory=list)
name: str = "modality_dropout"
```

```
def __post_init__(self):
    self._t0: float | None = None
```

```

def _in_drop(self, ts: float | None) -> bool:
    if ts is None:
        return False
    if self._t0 is None:
        self._t0 = ts
    rel = ts - self._t0
    return any(a <= rel < b for a, b in self.intervals_s)

def apply(self, item: StreamItem) -> StreamItem | None:
    modality = "video" if isinstance(item, FrameItem) else "audio"
    if modality == self.modality and self._in_drop(item.ts):
        return None
    return item

```

ConfidenceJitter — дрожание «качества» кадра через перекодирование *JPEG* с пониженным *quality* (имитация кратковременной потери резкости). Влияет на *quality.img_sharpness*, который считает *visual-detector*, и косвенно — на *adaptive gating*.

```

@dataclass
class ConfidenceJitter:
    """Дрожание качества кадра (имитация потери резкости через перекодирование JPEG)."""

    prob: float = 0.0
    low_quality: int = 25
    seed: int | None = None
    name: str = "confidence_jitter"

    def __post_init__(self):
        self._rng = random.Random(self.seed)

    def apply(self, item: StreamItem) -> StreamItem | None:
        if not isinstance(item, FrameItem) or self._rng.random() >= self.prob:
            return item
        import cv2
        buf = np.frombuffer(item.jpeg_bytes, dtype=np.uint8)
        img = cv2.imdecode(buf, cv2.IMREAD_COLOR)
        if img is None:
            return item
        ok, re = cv2.imencode(".jpg", img, [int(cv2.IMWRITE_JPEG_QUALITY), int(self.low_quality)])
        if not ok:
            return item
        return FrameItem(jpeg_bytes=re.tobytes(), seq=item.seq, width=item.width,
                        height=item.height, fps_nominal=item.fps_nominal, ts=item.ts,
                        meta={**item.meta, "jitter": "1"})

```